Topological Neural Operators

Anonymous Author(s)
Affiliation
Address
email

Abstract

We introduce **Topological Neural Operators** (TNOs), a principled framework for operator learning on cell complexes that lifts neural operators (NOs) from functions on points to topological domains. TNOs represent data as features defined on cells of varying dimension and model their interactions through *Discrete Exterior Calculus*, enabling explicit cross-dimensional coupling via gradient-, curl-, and divergence-type operators. The key design principle is to decouple *where* information flows—governed by fixed topological operators—from *how* it is transformed, which is learned, yielding models respecting the geometric support of physical quantities and exposing conservation and compatibility structure. We further propose **Hierarchical TNOs** (HTNOs), which incorporate learned coarse complexes to propagate long-range and topology-dependent information. Our framework subsumes existing NOs as a special case, providing a unified perspective on operator learning across discretizations. Across a suite of PDE benchmarks, TNOs and HTNOs achieve improved accuracy while exhibiting stronger physical consistency in structured settings.

1 Introduction

Modeling and predicting complex physical systems is a central challenge in science and engineering. Many systems, from fluid dynamics to electromagnetism and elasticity, are governed by *partial differential equations* (PDEs), whose solutions under varying parameters, geometries, and boundary conditions underpin applications such as digital twins, design optimization, and real-time simulation [6]. Classical numerical methods (e.g., finite elements, spectral solvers) are accurate but computationally costly, often requiring fine discretizations and multiple solves/iterations.

Operator learning addresses this by learning the solution itself: a map between infinite-dimensional function spaces that captures how PDE solutions depend on inputs such as coefficients, forcing terms, and geometry [16, 42, 41, 55]. *Neural operators* (NOs) [43] have emerged as a powerful instantiation of this idea, achieving strong performance and generalization across discretizations and resolutions via variants such as Fourier Neural Operators (FNOs) [47, 54] and their geometric or transformer-based extensions [35, 1]. These methods enable orders-of-magnitude speedups over classical solvers while maintaining high fidelity, establishing a scalable paradigm for scientific machine learning.

Despite these successes, existing NOs are fundamentally *point-centric*: they represent all physical quantities as functions defined on nodes, with interactions modeled via learned kernels or graph-based message passing. This abstraction neglects a key structural aspect of continuum physics: *physical quantities have geometric type*. Potentials live on vertices, circulations on edges, fluxes on faces, and densities in volumes; their interactions are governed by differential operators like gradient, curl, and divergence, that couple quantities across dimensions. In compatible numerical discretizations, these relationships are encoded directly through incidence structures, ensuring conservation laws and identities (e.g., $\mathrm{div}\,\mathrm{curl} = 0$) hold by construction[1]. By collapsing all quantities to nodes, current

[1]The boundary- of-boundary is zero: a boundary shape (like a compact/closed surface) has no boundary of its own.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

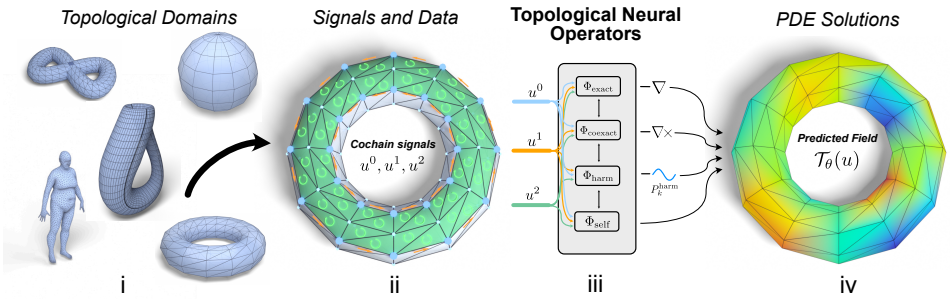


Figure 1: **Topological Neural Operators** operate on cell complexes (i) whose physical signals are cochains at multiple ranks (ii). A TNO layer (iii) couples ranks through fixed DEC operators (d^k, δ^k) and learns the rank-wise channel mixing in four blocks, signifying gradient, curl, harmonic, and self maps, producing a predicted PDE field on the complex (iv).

neural operators obscure this structure, limiting their ability to model multi-physics systems, enforce conservation and compatibility, and capture topological effects such as cycles and harmonic modes.

As a remedy, we introduce **Topological Neural Operators (TNOs)**, a principled framework for operator learning on *cell complexes*, the natural discrete domain for multi-dimensional physical systems (see Fig. 1). Our key design principle is to separate where information flows, governed by fixed topological operators, from how it is transformed, which is learned. Rather than representing data solely on points (or edges), TNOs operate on *cochains*, assigning features to cells of different dimensions (vertices, edges, faces, and higher-dimensional elements), and model their interactions using *Discrete Exterior Calculus* (DEC) [39]. This design explicitly encodes cross-dimensional structure: information flows between cells through fixed incidence-based operators, while learnable components act on feature transformations. As a result, TNOs (i) respect the geometric support of physical quantities, (ii) incorporate conservation and compatibility structure into the architecture, and (iii) enable explicit modeling of multi-degree PDE systems.

TNOs strictly generalize classical NOs: restricting the framework to 0-cochains recovers standard point-based models. To efficiently capture global and topology-dependent effects, we further introduce **Hierarchical TNOs** (HTNOs), which propagate information across learned coarse complexes while preserving the same cochain structure. This enables efficient modeling of long-range interactions and topological components associated with the domain. In brief, **our contributions are:**

- A **framework for operator learning on cell complexes**, extending neural operators to cochain-valued fields and enabling explicit cross-dimensional modeling via DEC.
- A **unifying perspective** encapsulating existing NOs as special cases of the proposed framework.
- **Hierarchical HTNOs** for efficient multi-scale propagation of global and topological information.
- **Controlled ablation studies** validating our design decisions, including PDEs that natively multiple-rank, demonstrating the inherent capability of TNOs at splitting physical signals across dimensions.

We validate our approach across synthetic and real PDE benchmarks. (H)TNOs achieve improved accuracy and generalization across meshes, while exhibiting stronger physical consistency in structured settings. We will make our implementation publicly available upon publication.

2 Background and the Language of Discrete Topology

Physical quantities are dimensional: scalars live on vertices, circulations on edges, fluxes on faces, densities in volumes. The laws relating them are statements about boundaries, orientations, and dimensional couplings; this is the language of discrete exterior calculus used throughout. We develop the necessary language in three steps. We refer the reader to [28, 27, 39, 29] for a rigorous treatment.

2.1 Topological Structures

Definition 2.1 (Regular Cell Complex)**.** *A* regular cell complex *is a finite set K together with a rank function* $\mathrm{rk} : K \to \mathbb{Z}_{\geq 0}$ *and a* face relation $\sigma \prec \tau$*, defined only when* $\mathrm{rk}(\tau) - \mathrm{rk}(\sigma) = 1$*, such that for any* $\tau, \rho \in K$ *with* $\mathrm{rk}(\tau) - \mathrm{rk}(\rho) = 2$*, exactly two elements* $\sigma \in K$ *satisfy both* $\rho \prec \sigma$ *and* $\sigma \prec \tau$*.*

Elements of K are called *cells*; those of rank k form the set K_k, with $n_k = |K_k|$ and $N = \max_\tau \mathrm{rk}(\tau)$. Rank-0 cells are vertices, rank-1 edges, rank-2 faces, and rank-3 volumes. This

structure is the right domain language for physics: quantities such as pressure, circulation, magnetic flux, and charge density live on cells of rank $0, 1, 2, 3$ respectively (Tab. 4). Every k-cell is a first-class computational entity able to exchange information with any incident cell. As we show in Sec. 3, the governing equations of Maxwell, Navier–Stokes, and elasticity are precisely statements about how quantities at different ranks interact, a structure graphs cannot represent and cell complexes encode natively. We shorten a regular cell complexes as RCC.

The face relation $\sigma \prec \tau$ records immediate incidence and determines differential operators. For every k-cell τ and $(k-1)$-cell σ one assigns an orientation sign $[\tau : \sigma] \in \{-1, 0, +1\}$, with $[\tau : \sigma] \neq 0$ iff $\sigma \prec \tau$. Assembling these into the *signed incidence matrix* $B_k \in \mathbb{R}^{n_{k-1} \times n_k}$ with $(B_k)_{\sigma\tau} = [\tau : \sigma]$, the diamond condition forces

$$B_k B_{k+1} = 0 \quad \forall\, k, \tag{1}$$

the discrete statement that **the boundary of a boundary is empty**, which underlies $\mathrm{curl}(\mathrm{grad}) = 0$ and $\mathrm{div}(\mathrm{curl}) = 0$, and is the structural foundation of everything that follows.

A *k-cochain* $u^k : K_k \to \mathbb{R}^{d_k}$ assigns a feature vector to each k-cell; the cochain space is $C^k(K; \mathbb{R}^{d_k}) \cong \mathbb{R}^{n_k \times d_k}$. The degree k is geometrically meaningful: it records the type of object on which the quantity lives, and assigning a field to the wrong degree breaks the structural laws governing it (Sec. A.2). The full multirank signal space is $C^\bullet = \bigoplus_{k=0}^{N} C^k$.

In practice, we also use more flexible Combinatorial Complexes [32, 31], which generalize graphs, RCCs, and hypergraphs by allowing cells at different ranks to be specified independently rather than constrained to arise as boundaries of higher-rank cells. We introduce these in Sec. 4.

2.2 Discrete Exterior Calculus

From the boundary matrices, one derives three families of operators that serve as the computational primitives of the Discrete Exterior Calculus (DEC) framework.

Discrete exterior derivative. The *discrete exterior derivative* $d^k = B_{k+1}^\top : C^k \to C^{k+1}$ lifts a k-cochain to a $(k+1)$-cochain by accumulating signed values over incidence relations. Concretely, d^0 is the discrete gradient (oriented differences across edges), d^1 is the discrete curl (oriented sums around faces), and d^2 is the discrete divergence-of-dual (oriented sums over bounding faces of a volume). The identity $d^{k+1} \circ d^k = 0$ follows directly from Eq. (1) and is the discrete counterpart of $\mathrm{curl}(\mathrm{grad}) = 0$ and $\mathrm{div}(\mathrm{curl}) = 0$.

Hodge operators and codifferential. Equipping each C^k with a positive-definite *Hodge star* $M_k \in \mathbb{R}^{n_k \times n_k}$ encoding geometric data such as edge lengths, face areas, and cell volumes, gives the formal M_k-adjoint of d^{k-1}, the *codifferential*, mapping downward in degree:

$$\delta^k = M_{k-1}^{-1} B_k M_k : C^k \to C^{k-1}, \tag{2}$$

While d^k is purely topological (determined by B_{k+1} alone), δ^k depends on geometry encoded in M_k.

Hodge Laplacian. Together, d^k and δ^k yield the *k-Hodge Laplacian*

$$\Delta_k = \underbrace{\delta^{k+1} \circ d^k}_{\Delta_k^\uparrow} + \underbrace{d^{k-1} \circ \delta^k}_{\Delta_k^\downarrow} : C^k \to C^k, \tag{3}$$

which is self-adjoint and positive semidefinite with respect to M_k. The decomposition into $\Delta_k^\uparrow$ and $\Delta_k^\downarrow$ is physically meaningful: $\Delta_k^\uparrow$ couples u^k upward to $(k+1)$-cells via d^k (curl-like coupling), while $\Delta_k^\downarrow$ couples u^k downward to $(k-1)$-cells via δ^k (divergence-like coupling). At $k = 0$, $\Delta_0 = \delta^1 d^0$ recovers the weighted graph Laplacian; at $k = 1$, Δ_1 simultaneously involves edges, vertices, and faces, an operator with no graph-level analog.

Hodge Decomposition (HD). The three operator families organize every cochain space into an orthogonal decomposition with respect to the M_k-inner product:

$$C^k(K; \mathbb{R}) = \underbrace{\mathrm{im}(d^{k-1})}_{\text{exact}} \oplus \underbrace{\ker(\Delta_k)}_{\text{harmonic}} \oplus \underbrace{\mathrm{im}(\delta^{k+1})}_{\text{coexact}}, \tag{4}$$

Exact cochains are potential-driven; coexact cochains are divergence-free; harmonic cochains are topologically constrained, with $\dim \ker(\Delta_k) = \beta_k$ counting k-dimensional holes.

HD is central to our TNO for two reasons: it characterizes what physical solutions look like (incompressible flows are coexact; conservative fields are exact; topological modes are harmonic), and it provides the DEC-compatible channels through which conservation and compatibility constraints can be preserved, , as we show in Sec. 4.1.

The complex K thus encodes in a single structure everything the framework needs: the domain $\{K_k\}_{k=0}^N$ stratified by dimension, the signal spaces $\{C^k(K;\mathbb{R}^{d_k})\}$ where physical fields live, the differential operators $d^k = B_{k+1}^\top$, $\delta^k = M_{k-1}^{-1} B_k M_k$, and $\Delta_k = \delta^{k+1} d^k + d^{k-1}\delta^k$ that move information between dimensions, and the conservation identity $B_k B_{k+1} = 0$ that encodes the algebraic backbone of discrete physical laws.

3 Discrete PDEs on Cell Complexes

The operators d^k, δ^k, Δ_k appear directly in the governing equations of physical systems, and most of these systems couple fields across more than one degree.

As in Sec. 2, let K be a finite cell complex of dimension N equipped with Hodge stars $\{M_k\}_{k=0}^N$. The de Rham complex governs the bulk of continuum physics on bounded domains, and discrete PDEs in this class take a uniform form. A *discrete PDE on* $(K, \{M_k\})$ *at degree* k is an equation

$$\mathcal{F}\big(u^k,\ d^k u^k,\ \delta^k u^k,\ \Delta_k u^k,\ a,\ f\big) = 0 \quad \text{in } C^k(K;\mathbb{R}^d), \tag{PDE}$$

where $u^k \in C^k(K;\mathbb{R}^d)$ is the unknown, $a \in C^\bullet(K;\mathbb{R}^p)$ encodes material parameters, $f \in C^k(K;\mathbb{R}^d)$ is a source, and $\mathcal{F}$ is a (possibly nonlinear) functional. Boundary conditions are imposed on a subcomplex $\partial K \subseteq K$: essential conditions fix the trace $u^k|_{\partial K}$, while natural conditions are encoded through the boundary-modified codifferential induced by $\{M_k\}$.

Most physical systems involve more than one degree at once: Maxwell couples a 1-form with a 2-form, Darcy couples a 0-form with a 1-form, Navier–Stokes in vorticity form couples 1- and 2-forms. A *multi-degree discrete PDE* is a system

$$\mathcal{F}_k\big(\{u^j\}_{j=0}^N,\ \{d^j u^j\}_j,\ \{\delta^j u^j\}_j,\ a,\ f\big) = 0 \quad \text{in } C^k(K;\mathbb{R}^{d_k}), \quad k = 0,\dots,N, \tag{mPDE}$$

with unknown $u \in C^\bullet(K;\mathbb{R}^{d_\bullet})$. The equation at degree k can depend on $d^{k-1}u^{k-1}$ (information from below, via $\Delta_k^\downarrow$-type coupling), on $\delta^{k+1}u^{k+1}$ (from above, via $\Delta_k^\uparrow$-type coupling), and on $\Delta_k u^k$. The coupling pattern is fixed by the cellular structure of K; the weights, by $\{M_k\}$. A learnable map that processes degrees independently, without d and δ connecting them, cannot represent this.

For time-dependent problems, the unknown is $u : [0,T] \to C^\bullet(K;\mathbb{R}^{d_\bullet})$ satisfying

$$\partial_t u^k(t) = \mathcal{A}_k\big(\{u^j(t)\}_{j=0}^N,\ \{d^j u^j(t)\}_j,\ \{\delta^j u^j(t)\}_j,\ a,\ f(t)\big), \qquad u(0) = u_0, \tag{Evol}$$

encompassing parabolic / hyperbolic problems, and fully coupled multi-physics systems. Discretizing in time yields one-step maps $u(t) \mapsto u(t+\Delta t)$, the natural targets for time-stepping operators.

Examples. In the Sec. A.3.1, we illustrate (Evol) with two standard systems whose fields live on different cochain degrees and couple through d and δ: discrete Maxwell- and wave-type couplings. In both cases, the dynamics are incidence-driven: fields on different cochain degrees are coupled by d and δ, and the associated constraints are preserved by the algebraic identities of the complex.

4 Topological Neural Operators (TNOs)

Each PDE admits a *solution operator* sending input data (initial conditions, sources, parameters) to the output field: for the wave system, a map $C^0 \times C^1 \to C^0 \times C^1$ advancing (u,p) in time; for Maxwell, a map $C^1 \times C^2 \to C^1 \times C^2$ advancing (E,B). Approximating such an operator $\mathcal{G}$ from data is the subject of operator learning [43]. Here $\mathcal{G}$ acts on tuples of cochains organized by degree, with its action mediated by d, δ, Δ on K. The following defines our class of learnable maps:

Definition 4.1 (Topological Neural Operators)**.** *Let K be a finite cell complex of dimension N with boundary matrices $\{B_k\}$ and Hodge stars $\{M_k\}_{k=0}^N$, inducing discrete operators $\{d^k\}$, $\{\delta^k\}$, $\{\Delta_k\}$. For $i = 1,\dots,m$ and $j = 1,\dots,n$, let $k_i, \ell_j \in \{0,\dots,N\}$ and $d_i, r_j \geq 1$, and define*

$$\mathcal{X}(K) = \prod_{i=1}^m C^{k_i}(K;\mathbb{R}^{d_i}), \qquad \mathcal{Y}(K) = \prod_{j=1}^n C^{\ell_j}(K;\mathbb{R}^{r_j}).$$

A Topological Neural Operator (TNO) *is a parameterized family* $\{\mathcal{T}_\theta^K : \mathcal{X}(K) \to \mathcal{Y}(K)\}_{\theta\in\Theta}$ *approximating a target operator* $\mathcal{G} : \mathcal{X}(K) \to \mathcal{Y}(K)$ *and satisfying:*

(P1) Cellular intrinsicness. *$\mathcal{T}_\theta^K$ is built from the cellular operators of K, including boundary / coboundary maps, Hodge stars, codifferentials, Hodge Laplacians, trace / restriction maps, pointwise nonlinearities, and learned channel-mixing maps shared across cells of the same degree. Thus it depends on incidence and metric data, not on a fixed grid or cell ordering.*

(P2) Multi-degree coupling. *Information may move between cochain degrees through boundary, coboundary, codifferential, Laplacian, and learned composed operators. Thus an output at one degree may depend on inputs at any other degree.*

(P3) Discretization transferability. *The weights θ are shared across discretizations. If K' refines K, then lifting to K' (prolonging cochains via $P_{K\to K'}$), applying the operator, and restricting back (via $R_{K'\to K}$) should agree with applying the operator on K:*

$$R_{K'\to K}\mathcal{T}_\theta^{K'} P_{K\to K'} x \approx \mathcal{T}_\theta^K x.$$

Remarks. (**P1**) makes the TNO intrinsic to the cellular discretization: incidence enters through the boundary maps, while geometric information enters through the Hodge stars, not through an extrinsic grid or an ordering of cells. (**P2**) is the key higher-order feature: signals may live on vertices, edges, faces, and higher cells, and the operator can couple these degrees. Such cross-degree coupling is not merely an architectural extension, but is imposed by the physics (e.g. Maxwell/wave PDEs in Sec. A.3.1). (**P3**) is the analogue of discretization consistency in NOs [43]: shared weights act across refinements.

The choice of degrees $\{k_i\}$ and $\{\ell_j\}$ in Dfn. 4.1 determines the operator type: degree-preserving ($C^k \to C^k$) for single-field PDEs; gradient-type ($C^0 \to C^1$) for potential-to-flux maps like Darcy or Fourier; divergence-type ($C^1 \to C^0$) for flux-to-source maps; and multi-degree ($C^0 \times C^1 \times C^2 \to C^0 \times C^1 \times C^2$) for coupled systems like Maxwell. We next detail how $\mathcal{T}_\theta$ is computed (*learned*).

4.1 Realization as a Topological Neural Network

Dfn. 4.1 specifies what a TNO must be (P1–P3); it does not say how to compute one. We now present one possible architecture that realizes these axioms as a topological neural network (TNN) [32].

Architecture. The TNO of Dfn. 4.1 is posed on an RCC K. For computation, it is convenient to lift K to a CC. A TNN realizing $\mathcal{T}_\theta^K : \mathcal{X}(K) \to \mathcal{Y}(K)$ has the form

$$\mathcal{T}_\theta^K = \mathrm{Dec}_\theta \circ \mathcal{L}_\theta^{(L-1)} \circ \cdots \circ \mathcal{L}_\theta^{(0)} \circ \mathrm{Enc}_\theta. \tag{5}$$

The encoder lifts the input cochains on K to hidden cochains $h_0^k \in C^k(\widetilde{K}; \mathbb{R}^{d_h})$ at every rank $k = 0, \ldots, \dim \widetilde{K}$. The incidence structure of $\widetilde{K}$ supplies the discrete operators used in the network. The topological layers $\mathcal{L}_\theta^{(\ell)}$ update these cochains on $\widetilde{K}$ while preserving their cochain-valued structure. The decoder reads the target degrees from the final hidden cochains and restricts to K.

The encoder and decoder bridge between K and $\widetilde{K}$. Material parameters and other coefficient fields enter as input cochains, which keep the architecture intrinsic to $(\widetilde{K}, \{M_k\})$ in the sense of (P1).

4.2 Topological layers

Each topological layer updates the hidden cochains on the lifted complex $\widetilde{K}$ while preserving their geometric type. Let $\mathbf{H} = (H^0, \ldots, H^N)$ with $H^k \in \mathbb{R}^{n_k \times d_h}$. The layer applies a learned nonlinear map ϕ_θ to a fixed block-structured operator that encodes the discrete exterior calculus of $\widetilde{K}$. As shown on the right, $\mathbf{H}_{\mathrm{out}}$ is defined through incidence matrices and Hodge stars of $\widetilde{K}$. Its tridiagonal sparsity is exact: equations at degree k couple only ranks $k \pm 1$. The trainable objects are the channel-mixing matrices W and transformations ϕ_θ. In the rigid DEC version, the transport maps themselves are fixed; in the copresheaf variant below, the incidence support remains fixed, but the fiber maps carried along incidences may be learned.

$$\mathbf{H}_{\mathrm{out}} = \phi_\theta\left(\begin{bmatrix} \Delta_0 & \delta^1 & 0 & \cdots \\ d^0 & \Delta_1 & \delta^2 & \cdots \\ 0 & d^1 & \Delta_2 & \ddots \\ \vdots & \vdots & \ddots & \ddots \end{bmatrix} W \star \mathbf{H}\right)$$

When only a single rank k is active, the block form reduces to (out-of-range terms omitted):

$$H_{\mathrm{out}}^k = \phi_\theta\Big(H^k,\ d^{k-1}H^{k-1}W_k^{\downarrow},\ \delta^{k+1}H^{k+1}W_k^{\uparrow},\ \Delta_k^{\uparrow}H^k W_k^{\Delta,\uparrow},\ \Delta_k^{\downarrow}H^k W_k^{\Delta,\downarrow}\Big). \tag{6}$$

Fig. 2 illustrates the template nature of TNOs: Maxwell fixes the active degrees and DEC routes, while the network learns how to mix the routed edge and face cochains (see Appx B). The construction admits a continuous *rigidity spectrum*. At the rigid extreme, the transport operators are exactly the DEC operators of $\widetilde{K}$. At the flexible extreme, each incidence $y \prec x$ carries a learnable fiber map $\rho_{y\to x}$, yielding twisted operators d_ρ, δ_ρ, $\Delta_{\rho,k}$ (copresheaf realization).

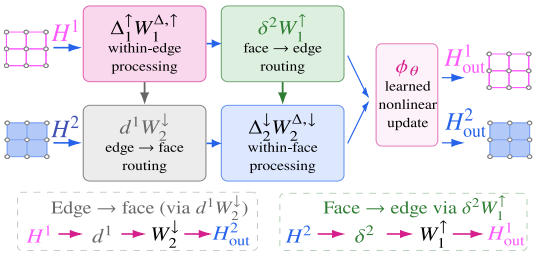


Figure 2: Visual TNO layer induced by Maxwell-type routing. Cross-rank transport is carried by d^1 and δ^2, while same-rank propagation is carried by the Hodge-Laplacian channels $\Delta^{\uparrow}$ and $\Delta^{\downarrow}$. The learned weights $W^{\bullet}$ and nonlinear update ϕ_θ determine how strongly the routed features are mixed.

Operator-learning interpretation.

The distinction from a generic CCNN layer [32] is that the support of spatial transport is prescribed by the DEC operators. Cross-rank coupling is carried by d and δ, while same-rank propagation is carried by $\Delta^{\uparrow}$ and $\Delta^{\downarrow}$. Thus, the architecture does not learn where information can flow; it learns how transported features are mixed through $W^{\bullet}$ and ϕ_θ. This is the architectural mechanism behind discretization transfer. Under compatible refinements and stable Hodge discretizations, the DEC operators d, δ, Δ approximate their continuum counterparts in the FEEC sense [3]. Since the same weights θ are shared across discretizations with support maps recomputed from each cell complex; the resulting family $\{\mathcal{T}_\theta^K\}_K$ is biased toward a common continuum rather than a discretization-specific message passing rule.

Linear realization of TNO. The simplest special case takes ϕ_θ to be linear and uses the Hodge decomposition channels directly. The rank-k linear TNO layer is

$$\mathcal{T}_{\theta,k}(h) = W_k^{\uparrow}\,\delta^{k+1}h^{k+1} + W_k^{\downarrow}\,d^{k-1}h^{k-1} + W_k^{\text{harm}}\,P_k^{\text{harm}}h^k + W_k^{\text{self}}\,h^k. \tag{7}$$

The four terms correspond to coexact, exact, harmonic, and local channels: $\mathrm{im}(\delta^{k+1})$, $\mathrm{im}(d^{k-1})$, $\ker(\Delta_k)$, and a residual self-channel. Thus, the layer separates the three topological components of a rank-k signal while keeping transport fixed by the DEC operators. The following propositions show how FNO and GNO arise as rank-0 specializations of the TNO framework.

Proposition 4.2 (FNO as a rank-0 spectral TNO [47])**.** *Let K_N be a uniform periodic cubical complex with $N_1 \times \cdots \times N_d$ vertices, and let $u \in C^0(K_N; \mathbb{R}^{c_{\text{in}}})$. Let $\mathcal{F}_N$ denote the discrete Fourier transform on 0-cochains. A linear FNO layer*

$$u \longmapsto Wu + \mathcal{F}_N^{-1} R_\theta \mathcal{F}_N u$$

is realized by a rank-0 spectral TNO layer whose same-rank spectral channel is diagonal in the Fourier basis of K_N.

Proposition 4.3 (GNO as a rank-0 graph-quadrature TNO [46])**.** *On a directed graph complex whose vertices are sample points and whose rank-1 cells encode the quadrature neighborhoods, the GNO update :*

$$u_i \mapsto Wu_i + \sum_{j\in\mathcal{N}(i)} \kappa_\theta(x_i, x_j) u_j \omega_j$$

is recovered as a rank-0 TNO with a learned incidence-supported kernel message $h^1(e_{ij}) = \kappa_\theta(x_i, x_j)u_j\omega_j$ on each directed rank-1 cell $e_{ij} : v_j \to v_i$, followed by target-incidence aggregation at each vertex. Proof is provided in Sec. C.2.

Hierarchical Realization of TNO (HTNO). The TNO construction extends naturally to a hierarchy of cell complexes. Let $K_0 \leftarrow K_1 \leftarrow \cdots \leftarrow K_L$, where $K_{\ell+1}$ is a coarsening of K_ℓ and $K_0 = K$. Each level carries cochain spaces $C^k(K_\ell)$ and discrete exterior derivatives $d_\ell^k : C^k(K_\ell) \to C^{k+1}(K_\ell)$. Levels are connected by degree-preserving transfer maps $R_\ell^k : C^k(K_\ell) \to C^k(K_{\ell+1})$ and $\Pi_\ell^k : C^k(K_{\ell+1}) \to C^k(K_\ell)$, chosen, when possible, to commute with the coboundary:

$$d_{\ell+1}^k R_\ell^k = R_\ell^{k+1} d_\ell^k, \qquad d_\ell^k \Pi_\ell^k = \Pi_\ell^{k+1} d_{\ell+1}^k. \tag{8}$$

This is analogous to the commuting-diagram condition of FEEC multigrid [4, 3]: exact, coexact, and harmonic components are not mixed by inter-level transfer, geometry is preserved across levels.

Table 1: Established steady-state benchmarks [48, 36] as used in [56]. We depict test relative L^1 (%) error. **Bold**: best; underline: second. † our runs with official implementations (see Sec. F.2).

	Ours		*Baselines*							
Dataset	HTNO	TNO	RIGNO-18 [56]	RIGNO-12 [56]	GAOT† [75]	MGN [61]	Geo-FNO [48]	FNO DSE [52]	GINO [49]	UPT [1]
Poisson-Gauss	1.30	**1.03**	2.26	2.52	1.39	30.9	8.16	2.27	7.57	48.4
Airfoil	1.21	1.11	**1.00**	1.09	3.12	10.1	4.48	1.99	2.00	45.7
Elasticity	**1.70**	2.73	4.31	4.63	4.07	11.9	5.53	4.81	4.38	12.6

Table 2: Additional steady-state benchmarks: test relative L^1 (%). †Our reproduction; **bold**: best, underline: second. EmmiWing is given per-variable (p_s, $\tau_{x,y,z}$) and aggregate (Agg).

(a) Compressible airfoils and PCS [75]

Dataset	HTNO	TNO	RIGNO†	GAOT†
NACA0012	4.53	**3.77**	5.09	12.13
NACA2412	4.73	**3.92**	5.73	12.25
RAE2822	4.55	**3.92**	4.81	12.98
PCS	**0.52**	3.09	7.11	0.72

(b) EmmiWing [58].

Model	p_s	τ_x	τ_y	τ_z	Agg.
HTNO	**0.24**	**3.94**	**5.32**	**5.37**	**2.41**
Transolver†	0.28	4.14	5.52	5.57	2.55
Transformer†	0.30	4.21	5.57	6.13	2.60
PointNet†	0.29	4.24	5.75	5.90	2.62
RIGNO-18†	0.29	4.46	6.02	6.30	2.78

An HTNO arranges TNO blocks as a learned V-cycle for de Rham complexes [37, 71]: TNO layers on K_ℓ are Hodge-compatible smoothers [38], with an additive coarse-grid correction transferred via R_ℓ^k, Π_ℓ^k that come from a partition of K_ℓ — either fixed (k-means) or learned end-to-end as a soft Voronoi with trainable centroids. The operator class is unchanged: it is still a TNO at every level — (P1)–(P3) hold level-wise — with fine blocks capturing local cross-degree physics and coarse blocks propagating long-range information. Sec. D gives the V-cycle equation and implementation.

5 Experiments

We evaluate TNO and HTNO on a broad span of steady-state PDEs over irregular 2D and 3D meshes: Poisson's equation with Gaussian and multiscale-sinusoidal sources, hyper-elastic deformations of variable domains, compressible flow past 2D airfoils spanning subsonic, transonic, and supersonic regimes, and large-scale 3D wing-surface aerodynamics under compressible RANS in the transonic regime. These come from three established public suites [56, 75, 58], with mesh sizes ranging from $\sim$1K to several hundred K nodes. We complement these with two controlled studies of our own: an anisotropic-Darcy experiment with per-face random tensor orientation that isolates how TNO natively ingests higher rank signals (Sec. 5.3), and a component ablation on synthetic topologies (Darcy with reaction; conservative advection–diffusion) that disentangles the harmonic-basis input and sheaf transport (Sec. 5.4). Datasets, baselines, training protocol, full result tables, and per-variable analyses are deferred to Sec. F. Implementations are in JAX and run on GH200 chips (96GB VRAM).

5.1 Steady-State Benchmarks on Irregular Geometry

Poisson-Gauss, Airfoil, Elasticity (Tab. 1). We evaluate on three established steady-state benchmarks assembled by [56]: Poisson-Gauss (PG, fixed grid; from [36]), Airfoil Flow (AF, variable geometry), and Elasticity (variable; both from [48]). We compare against the seven baselines reported in [56] (see Tab. 1. Both TNO and HTNO outperform all seven on PG and Elasticity; TNO is competitive with RIGNO-18 on AF, which fixes the far-field at M_∞=0.8, α=0° and varies only the airfoil geometry [48], so methods saturate near the discretization noise floor; the GAOT airfoils (Tab. 2a) genuinely sweep $M \in [0.5, 1.4]$ and $\alpha \in [0.5°, 5.0°]$ per sample [75], crossing the shock-formation boundary (see Sec. G.2).

Compressible airfoils and Poisson-with-sines (Tab. 2a). We additionally evaluate on the four single-file steady-state datasets [75]: three compressible-airfoil sets (NACA0012/2412, RAE2822) on 8K-node meshes, spanning subsonic to supersonic regimes, and Poisson-with-sines on a fixed 16K-node mesh (PCS). On all three airfoils, both TNO and HTNO clearly beat RIGNO and GAOT. This can be attributed to the elliptic characteristics that dominate steady compressible flow at subsonic and transonic conditions (smooth pressure/velocity outside shocks), which align naturally with the Hodge / harmonic-basis inductive biases that characterize TNOs. PCS is the converse setting: a fixed mesh with a smooth, multiscale-sinusoidal Poisson source, for which GAOT's structured-latent

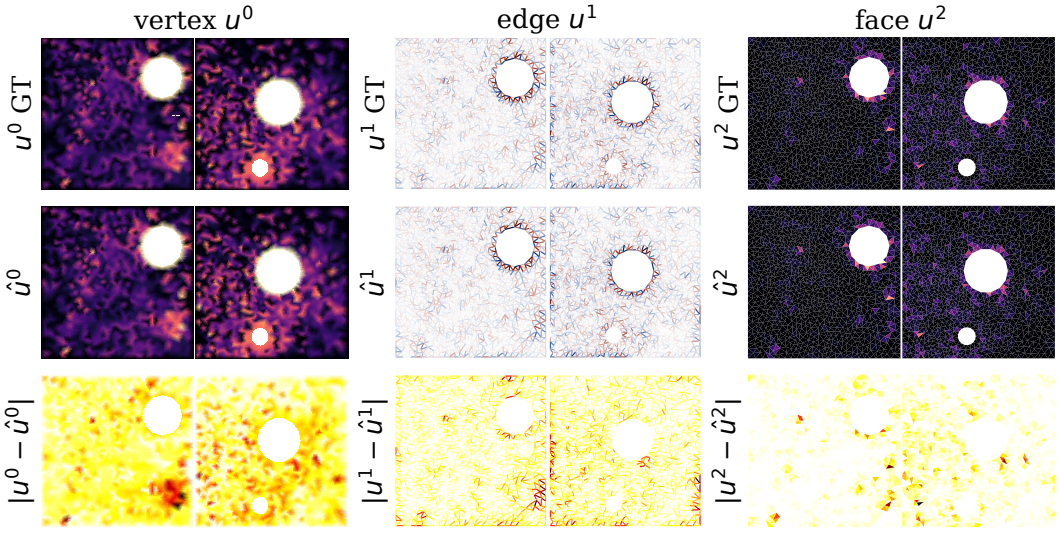


Figure 3: **Qualitative results for Anisotropic Darcy** with per-face random tensor orientations. Coupled signals require disentangling physical quantities at multiple topological ranks simultaneously.

encoder is well matched, beating RIGNO by an order of magnitude. Nevertheless, the HTNO, which combines local and global interactions, yields strong performance (**0.52** vs. 0.72).

5.2 Large-Scale Surface PDEs: EmmiWing

EmmiWing [58] comprises 3D wing surfaces (raw meshes 244K–426K nodes; variable per sample) with four output channels (surface pressure p_s plus three wall-shear-stress components $\tau_{x,y,z}$; official 25,674/999/2,992 split). Following the AB-UPT training regime [2], we train on 65K shared FPS query points per sample with 16,384-node random subsampling per step, and evaluate uniformly on the 65K FPS subset rather than the full raw mesh; see Sec. F.3 for the full protocol. HTNO leads aggregate L^1 at 2.41%, beating Transolver [76] (2.55%), our matched-budget self-attention Transformer (2.60%) and PointNet (2.62%), and RIGNO-18 (2.78%); HTNO additionally converges in $\sim 6\times$ fewer epochs than RIGNO-18. TNO is subsumed by the hierarchical HTNO, which models long-range interactions without per-sample Hodge decompositions on large resampled geometries.

5.3 Anisotropic Darcy with Per-Face Random Tensor Orientation

Setup. TNOs naturally accommodate higher-rank signals. To validate this topology-aware inductive bias, we simulate the following anisotropic Darcy PDE on 100 triangulated planar square domains $[-1, 1]^2$ with $K \in \{1, 2\}$ randomly placed circular holes:

$$-\nabla \cdot \big(K(x)\, \nabla u\big) = f, \qquad u|_{\partial\Omega_{\text{outer}}} = 0, \quad u|_{\partial\Omega_{\text{hole},k}} = g_k \sim \mathcal{U}[0.5, 1.5],$$

The diffusivity $K(x)$ is a piecewise-constant face-valued symmetric tensor with eigenvalues $(k_\parallel, k_\perp) = (4, 1)$ and a per-face principal axis $\phi_t \sim \mathcal{U}[0, \pi)$ *drawn iid per triangle and fixed across all samples on a given mesh*, so that the rank-2 orientation field $\cos(2\phi_t)$ has unit-order variance on every face, but *vanishing population mean*. **Projection onto vertices is therefore lossy by construction:** averaging $\cos(2\phi_t)$ over incident faces concentrates to ≈ 0 at every vertex regardless of the underlying K. This isolates the effect of *how* the orientation is consumed: an architecture that ingests it at its native rank 2 retains operator information that any architecture isolated to vertices discards. We compare TNO (rank-2, harmonic basis, sheaf transport) against an MPNN baseline at matched and parameter-matched width, each consuming the orientation field in three ways: not at all (vertex inputs only), as a vertex-projected channel, or natively at rank 2. PDE simulation, training recipe, mesh construction, and face-supervision cells are in Sec. F.4.

Results. Two distinct effects emerge (see Tab. 3a). **(i) Architecture, $\sim$4–5 pp, independent of how orientation is consumed.** TNO beats MPNN by 4.58 pp on vertex inputs and by 3.68 pp on projected inputs; even at parameter parity. The harmonic-basis channel resolves the spectral signature of the anisotropic operator, which vanilla message-passing cannot. **(ii) Native rank-2 ingestion, an additional $\approx$0.5 pp.** On the substrate where vertex projection is provably lossy, TNO with native

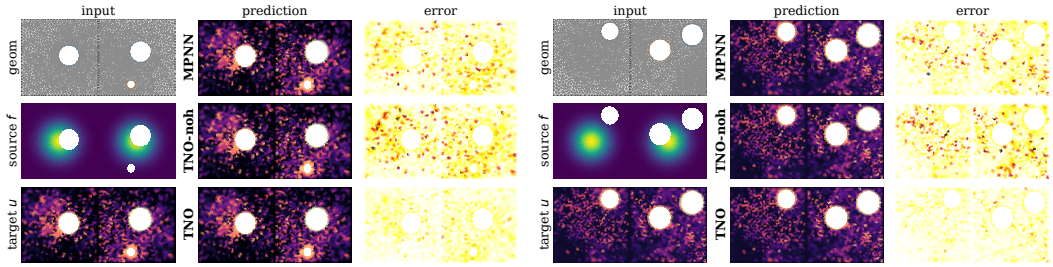


Figure 4: Qualitative ablation across three models: Vanilla MPNN vs. TNO (no harmonics) vs. TNO with harmonics / copresheaves on Anisotropic Darcy (left) and Advection Diffusion right), for which we depict model inputs and targets (col. 1), model predictions (col. 2), and model errors (col. 3).

Table 3: Rank-input and component studies: test relative error (%); lower is better, **bold**: best, underline: second; *param-matched MPNN (h=272, $\sim$5.35M $\approx$ TNO 5.0M).

(a) Anisotropic Darcy with per-face random tensor orientation (Sec. 5.3); L^1 / L^2.

Variant	L^1	L^2
MPNN (h=192), vertex	9.97	7.03
MPNN (h=192), projected	9.10	6.52
MPNN (h=272)*, projected	7.59	5.28
TNO, vertex	5.39	3.85
TNO, projected	5.42	3.66
TNO, native rank-2	**4.88**	**3.32**

(b) TNO component ablation on synthetic topologies (Sec. 5.4); L^1 per PDE.

Variant	Darcy	Adv.-Diff.
MPNN (h=192)	8.87	9.78
MPNN (h=272)*	7.29	7.66
TNO	9.44	10.68
TNO + harm. basis	5.16	5.77
TNO + copresheaf	11.64	13.05
TNO + copresheaf + harm.	**4.87**	**5.20**

rank-2 orientation beats TNO with projected orientation by 0.54 pp ($\sim$ 10% relative), while the projected and vertex-only variants are within 0.03 pp of each other, confirming that the projected channel carries no operator information, as demanded by the iid-ϕ construction. The two effects are additive: the architecture earns $\sim$5 pp; native rank-2 ingestion yields another $\sim$0.5 pp improvement.

5.4 Ablations

Setup. We isolate the contribution of TNO's harmonic-basis input and sheaf transport on two scalar PDEs whose flux carries non-trivial curl on the same mesh family as Sec. 5.3: Darcy with reaction ($-\nabla\cdot(K\nabla u) + \sigma u = f$) and conservative advection–diffusion. Per-sample variability is restricted to the per-hole Dirichlet BC values, so the topology is the only axis that varies across samples on a fixed mesh. All variants share a common backbone (hidden dim 192; run 300 epochs); MPNN baselines at matched width and parameter count bracket the comparison.

Results. Disabling the harmonic basis yields a loss of 6.8 pp on Darcy and 7.9 pp on Adv.-Diff (see Tab. 3b). Sheaf transport and the harmonic basis are synergistic, not additive: with harmonic on, sheaf helps; with harmonic off, sheaf actively hurts (Darcy $9.44 \rightarrow 11.64$). The complete TNO wins on both PDEs, beating the param-matched MPNN by $\sim$33% (see Fig. 4). This implies harmonic and sheaf components are an effective inductive bias, particularly for PDEs with non-trivial curl.

6 Conclusion

We introduced **TNOs**, a topological framework for operator learning on cell complexes. Fields are represented as cochains on vertices, edges, faces, and volumes, and their interactions are routed by DEC operators. Thus topology determines where information flows, while learning determines how transported features are transformed. This yields architectures suited to multi-physics coupling, irregular geometries, and discretization transfer, with existing neural operators arising as special cases. Empirically, TNOs and HTNOs perform strongly across irregular geometries and higher-rank PDE systems, improving physical consistency. This points toward scientific foundation models that can learn operators on structured spaces, unifying geometry, physics, and computation.

Limitations and future work. This work leaves several challenges open: scaling to dynamic, adaptive, and very large domains; developing approximation, stability, and convergence theory, clarifying the role of harmonic and sheaf-based channels; and extending TNOs to multiscale systems, evolving manifolds, inverse problems, and gauge- or symmetry-equivariant operator families.

References

[1] Benedikt Alkin, Andreas Fürst, Simon Schmid, Lukas Gruber, Markus Holzleitner, and Johannes Brandstetter. Universal physics transformers: A framework for efficiently scaling neural operators. In *Adv. Neural Inform. Process. Syst.*, 2024.

[2] Benedikt Alkin, Maurits Bleeker, Richard Kurle, Tobias Kronlachner, Reinhard Sonnleitner, Matthias Dorfer, and Johannes Brandstetter. AB-UPT: Scaling neural CFD surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers. *Trans. Mach. Lear. Resea.*, 2025. arXiv:2502.09692.

[3] Douglas N. Arnold. *Finite Element Exterior Calculus*. SIAM, Philadelphia, PA, 2018. doi: 10.1137/1.9781611975543.

[4] Douglas N Arnold, Richard S Falk, and Ragnar Winther. Finite element exterior calculus, homological techniques, and applications. *Acta Numerica*, 15:1–155, 2006.

[5] Douglas N Arnold, Richard S Falk, and Ragnar Winther. Finite element exterior calculus: from Hodge theory to numerical stability. *Bulletin of the American Mathematical Society*, 47(2): 281–354, 2010.

[6] Kamyar Azizzadenesheli, Nikola Kovachki, Zongyi Li, Miguel Liu-Schiaffini, Jean Kossaifi, and Anima Anandkumar. Neural operators for accelerating scientific simulations and design. *Nature Reviews Physics*, 6(5):320–328, 2024.

[7] Igor A. Baratta, Joseph P. Dean, Jørgen S. Dokken, Michal Habera, Jack S. Hale, Chris N. Richardson, Marie E. Rognes, Matthew W. Scroggs, Nathan Sime, and Garth N. Wells. DOLFINx: The next generation FEniCS problem solving environment. Zenodo, 2023.

[8] Nathan Bell and Anil N. Hirani. PyDEC: Software and algorithms for discretization of exterior calculus. *ACM Transactions on Mathematical Software*, 39(1):3:1–3:41, 2012. doi: 10.1145/2382585.2382588.

[9] Kaushik Bhattacharya, Bamdad Hosseini, Nikola B Kovachki, and Andrew M Stuart. Model reduction and neural networks for parametric PDEs. *The SMAI Journal of Computational Mathematics*, 7:121–157, 2021.

[10] Ginestra Bianconi. The topological Dirac equation of networks and simplicial complexes. *Journal of Physics: Complexity*, 2(3):035022, 2021.

[11] Cristian Bodnar, Fabrizio Frasca, Nina Otter, Yuguang Wang, Pietro Liò, Guido Montúfar, and Michael Bronstein. Weisfeiler and lehman go cellular: CW networks. In *Adv. Neural Inform. Process. Syst.*, 2021.

[12] Cristian Bodnar, Fabrizio Frasca, Yuguang Wang, Nina Otter, Guido Montúfar, Pietro Liò, and Michael Bronstein. Weisfeiler and lehman go topological: Message passing simplicial networks. In *Int. Conf. Mach. Lear.*, 2021.

[13] Cristian Bodnar, Francesco Di Giovanni, Benjamin Chamberlain, Pietro Lio, and Michael Bronstein. Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in gnns. *Advances in Neural Information Processing Systems*, 35:18527–18541, 2022.

[14] Alain Bossavit. Whitney forms: A class of finite elements for three-dimensional computations in electromagnetism. *IEE Proceedings A (Physical Science, Measurement and Instrumentation, Management and Education, Reviews)*, 135(8):493–500, 1988.

[15] Alain Bossavit. *Computational Electromagnetism: Variational Formulations, Complementarity, Edge Elements*. Academic Press, 1998.

[16] Nicolas Boullé and Alex Townsend. A mathematical guide to operator learning. In *Handbook of Numerical Analysis*, volume 25, pages 83–125. Elsevier, 2024.

[17] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL `http://github.com/jax-ml/jax`.

[18] James H Bramble, Joseph E Pasciak, and Jinchao Xu. The analysis of multigrid algorithms with nonnested spaces or noninherited quadratic forms. *Mathematics of Computation*, 56(193):1–34, 1991.

[19] William L Briggs, Van Emden Henson, and Steve F McCormick. *A multigrid tutorial*. SIAM, 2000.

[20] Andrey Bryutkin, Jiahao Huang, Zhongying Deng, Guang Yang, Carola-Bibiane Schönlieb, and Angelica I Aviles-Rivero. Hamlet: Graph transformer neural operator for partial differential equations. In *International Conference on Machine Learning*, pages 4624–4641. PMLR, 2024.

[21] Lucille Calmon, Michael T. Schaub, and Ginestra Bianconi. Higher-order signal processing with the Dirac operator. In *56th Asilomar Conference on Signals, Systems, and Computers*, pages 925–929. IEEE, 2022. doi: 10.1109/IEEECONF56349.2022.10052062.

[22] Edoardo Calvello, Nikola B Kovachki, Matthew E Levine, and Andrew M Stuart. Continuum attention for neural operators. *Journal of Machine Learning Research*, 26(300):1–52, 2025.

[23] Gengxiang Chen, Xu Liu, Qinglu Meng, Lu Chen, Changqing Liu, and Yingguang Li. Learning neural operators on riemannian manifolds. *National Science Open*, 3(6):20240001, 2024.

[24] Chaoran Cheng and Jian Peng. Equivariant neural operator learning with graphon convolution. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, pages 61960–61984, 2023.

[25] Pengcheng Cheng. Gauge-equivariant intrinsic neural operators for geometry-consistent learning of elliptic pde maps. *arXiv preprint arXiv:2603.14734*, 2026.

[26] Jae Choi, Yuzhou Chen, Huikyo Lee, Hyun Kim, and Yulia R. Gel. SNN-PDE: Learning dynamic PDEs from data with simplicial neural networks. In *AAAI*, pages 11561–11569, 2024.

[27] Keenan Crane. Discrete differential geometry: An applied introduction. `https://www.cs.cmu.edu/~kmcrane/Projects/DDG/paper.pdf`, 2018. Lecture notes, Carnegie Mellon University.

[28] Keenan Crane, Fernando de Goes, Mathieu Desbrun, and Peter Schröder. Digital geometry processing with discrete exterior calculus. In *ACM SIGGRAPH 2013 Courses*, pages 1–126. 2013.

[29] Mathieu Desbrun, Anil N Hirani, Melvin Leok, and Jerrold E Marsden. Discrete exterior calculus. *arXiv preprint math/0508341*, 2005.

[30] Somdatta Goswami, Aniruddha Bora, Yue Yu, and George Em Karniadakis. Physics-informed deep neural operator networks. In *Machine learning in modeling and simulation: methods and applications*, pages 219–254. Springer, 2023.

[31] Mustafa Hajij, Ghada Zamzmi, Theodore Papamarkou, Aldo Guzmán-Sáenz, Tolga Birdal, and Michael T. Schaub. Combinatorial complexes: bridging the gap between cell complexes and hypergraphs. In *Asilomar Conference on Signals, Systems, and Computers*, pages 799–803. IEEE, 2023.

[32] Mustafa Hajij, Ghada Zamzmi, Theodore Papamarkou, Nina Miolane, Aldo Guzmán-Sáenz, Karthikeyan Natesan Ramamurthy, Tolga Birdal, Tamal K. Dey, Soham Mukherjee, Shreyas N. Samaga, Neal Livesay, Robin Walters, Paul Rosen, and Michael T. Schaub. Topological deep learning: Going beyond graph data. *arXiv preprint arXiv:2206.00606*, 2023.

[33] Mustafa Hajij, Lennart Bastian, Sarah Osentoski, Hardik Kabaria, John L Davenport, Sheik Dawood, Balaji Cherukuri, Joseph G Kocheemoolayil, Nastaran Shahmansouri, Adrian Lew, Theodore Papamarkou, and Tolga Birdal. Copresheaf topological neural networks: A generalized deep learning framework. In *Adv. Neural Inform. Process. Syst.*, 2025.

[34] Jakob Hansen and Thomas Gebhart. Sheaf neural networks. In *NeurIPS 2020 Workshop on Topological Data Analysis and Beyond*, 2020. arXiv:2012.06333.

[35] Zhongkai Hao, Zhengyi Wang, Hang Su, Chengyang Ying, Yinpeng Dong, Songming Liu, Ze Cheng, Jian Song, and Jun Zhu. Gnot: A general neural operator transformer for operator learning. In *International conference on machine learning*, pages 12556–12569. PMLR, 2023.

[36] Maximilian Herde, Bogdan Raonić, Tobias Rohner, Roger Käppeli, Roberto Molinaro, Emmanuel de Bézenac, and Siddhartha Mishra. Poseidon: Efficient foundation models for PDEs. *Advances in Neural Information Processing Systems*, 37:72525–72624, 2024.

[37] Ralf Hiptmair. Multigrid method for Maxwell's equations. *SIAM Journal on Numerical Analysis*, 36(1):204–225, 1998.

[38] Ralf Hiptmair. Finite elements in computational electromagnetism. *Acta Numerica*, 11:237–339, 2002.

[39] Anil Nirmal Hirani. *Discrete Exterior Calculus*. PhD thesis, California Institute of Technology, 2003.

[40] James M Hyman, Mikhail Shashkov, and Stanly Steinberg. The numerical solution of diffusion problems in strongly heterogeneous non-isotropic materials. *Journal of Computational Physics*, 132(1):130–148, 1997.

[41] Anran Jiao, Haiyang He, Rishikesh Ranade, Jay Pathak, and Lu Lu. One-shot learning for solution operators of partial differential equations. *Nature Communications*, 16(1):8386, 2025.

[42] Nikola B Kovachki, Samuel Lanthaler, and Andrew M Stuart. Operator learning: Algorithms and analysis. *Handbook of Numerical Analysis*, 25:419–467, 2024.

[43] Nikolas Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *J. Mach. Learn. Res.*, 24(89):1–97, 2023.

[44] Samuel Leventhal, Attila Gyulassy, Valerio Pascucci, and Mark Heimann. Modeling hierarchical topological structure in scientific images with graph neural networks. In *2023 IEEE International Conference on Image Processing (ICIP)*, pages 2995–2999. IEEE, 2023.

[45] Zijie Li, Kazem Meidani, and Amir Barati Farimani. Transformer for partial differential equations' operator learning. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL `https://openreview.net/forum?id=EPPqt3uERT`.

[46] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Graph kernel network for partial differential equations. In *ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations*, 2020. arXiv:2003.03485.

[47] Zongyi Li, Nikolas Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *ICLR*, 2021.

[48] Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, and Anima Anandkumar. Fourier neural operator with learned deformations for PDEs on general geometries. *J. Mach. Learn. Res.*, 24 (388):1–26, 2023.

[49] Zongyi Li, Nikola Kovachki, Chris Choy, Boyi Li, Jean Kossaifi, Shourya Otta, Mohammad Amin Nabian, Maximilian Stadler, Christian Hundt, Kamyar Azizzadenesheli, and Anima Anandkumar. Geometry-informed neural operator for large-scale 3D PDEs. In *Adv. Neural Inform. Process. Syst.*, 2023.

[50] Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations. *ACM/IMS Journal of Data Science*, 1(3):1–27, 2024.

[51] Yunfeng Liao, Yangxin Wu, and Xiucheng Li. Boundary-value PDEs meet higher-order differential topology-aware GNNs. In *Advances in Neural Information Processing Systems*, 2025.

[52] Levi E. Lingsch, Mike Yan Michelis, Emmanuel De Bezenac, Sirani M. Perera, Robert K. Katzschmann, and Siddhartha Mishra. Beyond regular grids: Fourier-based neural operators on arbitrary domains. In *Int. Conf. Mach. Lear.*, 2024.

[53] Konstantin Lipnikov, Gianmarco Manzini, and Mikhail Shashkov. Mimetic finite difference method. *Journal of Computational Physics*, 257:1163–1227, 2014.

[54] Ning Liu, Siavash Jafarzadeh, and Yue Yu. Domain agnostic fourier neural operators. *Advances in neural information processing systems*, 36:47438–47450, 2023.

[55] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3):218–229, 2021.

[56] Sepehr Mousavi, Shizheng Wen, Levi Lingsch, Maximilian Herde, Bogdan Raonic, and Siddhartha Mishra. Rigno: A graph-based framework for robust and accurate operator learning for pdes on arbitrary domains. In *Adv. Neural Inform. Process. Syst.*, 2025.

[57] Jean-Claude Nédélec. Mixed finite elements in $\mathbb{R}^3$. *Numerische Mathematik*, 35(3):315–341, 1980.

[58] Fabian Paischer, Leo Cotteleer, Yann Dreze, Richard Kurle, Dylan Rubini, Maurits Bleeker, Tobias Kronlachner, and Johannes Brandstetter. Going with the speed of sound: Pushing neural surrogates into highly-turbulent transonic regimes. In *NeurIPS 2025 Workshop on Machine Learning and the Physical Sciences*, 2025. arXiv:2511.21474.

[59] Theodore Papamarkou, Tolga Birdal, Michael M. Bronstein, Gunnar E. Carlsson, Justin Curry, Yue Gao, Mustafa Hajij, Roland Kwitt, Pietro Liò, Paolo Di Lorenzo, Vasileios Maroulas, Nina Miolane, Farzana Nasrin, Karthikeyan Natesan Ramamurthy, Bastian Rieck, Simone Scardapane, Michael T. Schaub, Petar Veličković, Bei Wang, Yusu Wang, Guowei Wei, and Ghada Zamzmi. Position: Topological deep learning is the new frontier for relational learning. In *Int. Conf. Mach. Lear.*, volume 235 of *Proceedings of Machine Learning Research*, pages 39529–39555, 2024.

[60] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[61] Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W. Battaglia. Learning mesh-based simulation with graph networks. In *ICLR*, 2021.

[62] Lenka Ptáčková and Luiz Velho. A simple and complete discrete exterior calculus on general polygonal meshes. *Computer Aided Geometric Design*, 88:102002, 2021.

[63] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *CVPR*, 2017.

[64] Blaine Quackenbush and PJ Atzberger. Geometric neural operators (gnps) for data-driven deep learning in non-euclidean settings. *Machine Learning: Science and Technology*, 5(4):045033, 2024. URL `https://doi.org/10.1088/2632-2153/ad8980`.

[65] Bogdan Raonic, Roberto Molinaro, Tim De Ryck, Tobias Rohner, Francesca Bartolucci, Rima Alaifari, Siddhartha Mishra, and Emmanuel de Bézenac. Convolutional neural operators for robust and accurate learning of PDEs. In *Adv. Neural Inform. Process. Syst.*, 2023.

[66] Michael T Schaub, Austin R Benson, Paul Horn, Gabor Lippner, and Ali Jadbabaie. Random walks on simplicial complexes and the normalized hodge 1-laplacian. *SIAM Review*, 62(2): 353–391, 2020.

[67] Dmitriy Smirnov and Justin Solomon. HodgeNet: Learning spectral geometry on triangle meshes. *ACM Transactions on Graphics (SIGGRAPH)*, 40(4), 2021.

[68] Alasdair Tran, Alexander Mathews, Lexing Xie, and Cheng Soon Ong. Factorized fourier neural operators. In *The Eleventh International Conference on Learning Representations*, 2023. URL `https://openreview.net/forum?id=tmIiMPl4IPa`.

[69] Nathaniel Trask, Andy Huang, and Xiaozhe Hu. Enforcing exact physics in scientific machine learning: A data-driven exterior calculus on graphs. *Journal of Computational Physics*, 456: 110969, 2022.

[70] Tapas Tripura and Souvik Chakraborty. Wavelet neural operator for solving parametric partial differential equations in computational mechanics problems. *Computer Methods in Applied Mechanics and Engineering*, 404:115783, 2023.

[71] Ulrich Trottenberg, Cornelis W Oosterlee, and Anton Schüller. *Multigrid*. Academic Press, 2000.

[72] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Adv. Neural Inform. Process. Syst.*, 2017.

[73] Francesco Viganò, Tolga Birdal, Michael T Schaub, and Mauricio Barahona. Root-to-leaf path random walks, normalized hodge laplacians, and cheeger inequalities on simplicial complexes. *arXiv preprint arXiv:2604.27241*, 2026.

[74] Hrishikesh Viswanath, Md Ashiqur Rahman, Abhijeet Vyas, Andrey Shor, Beatriz Medeiros, Stephanie Hernandez, Suhas Eswarappa Prameela, and Aniket Bera. Neural operator: Is data all you need to model the world? an insight into the paradigm of data-driven scientific ML. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026. doi: 10.1109/TPAMI.2026.3682604.

[75] Shizheng Wen, Arsh Kumbhat, Levi Lingsch, Sepehr Mousavi, Yizhou Zhao, Praveen Chandrashekar, and Siddhartha Mishra. Geometry aware operator transformer as an efficient and accurate neural surrogate for PDEs on arbitrary domains. In *Adv. Neural Inform. Process. Syst.*, 2025. arXiv:2505.18781.

[76] Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for PDEs on general geometries. In *Int. Conf. Mach. Lear.*, 2024.

[77] Minglang Yin, Nicolas Charon, Ryan Brody, Lu Lu, Natalia Trayanova, and Mauro Maggioni. A scalable framework for learning the geometry-dependent solution operators of partial differential equations. *Nature Computational Science*, 4(12):928–940, 2024.

[78] Seungwoo Yoo, Kyeongmin Yeo, Jisung Hwang, and Minhyuk Sung. Neural Green's functions. In *Adv. Neural Inform. Process. Syst.*, 2025.

[79] Weiheng Zhong and Hadi Meidani. Physics-informed geometry-aware neural operator. *Computer Methods in Applied Mechanics and Engineering*, 434:117540, 2025.

Topological Neural Operators

Appendix

Table of Contents

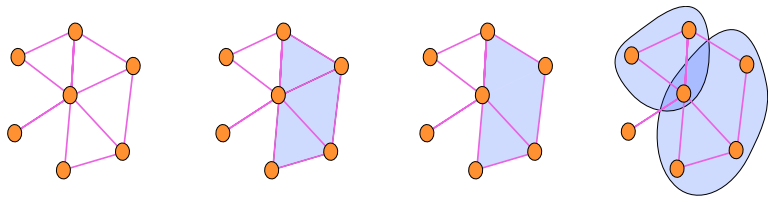
A Background

A.1 Combinatorial Complexes

Combinatorial complexes generalize simplicial and cell complexes as well as hypergraphs and act as a unifying topological structure:

Definition A.1 (Combinatorial complex [32, 31])**.** *A* combinatorial complex *(CC) is a triple* $(S, \mathcal{X}, \mathrm{rk})$ *consisting of a set* S *of entities, a subset* $\mathcal{X} \subseteq \mathcal{P}(S) \setminus \{\emptyset\}$ *of cells, and a rank function* $\mathrm{rk} : \mathcal{X} \to \mathbb{Z}_{\geq 0}$ *satisfying* $\mathrm{rk}(\{v\}) = 0$ *for all* $v \in S$ *and* $\mathrm{rk}(x) \leq \mathrm{rk}(y)$ *whenever* $x \subseteq y$*.*

Lifting an RCC K to a CC $\widetilde{K}$ retains the cochains of K but may carry additional cells–higher-order groupings or augmented neighborhoods–to enrich the information flow. The lift does not change the target operator $\mathcal{G}$ or the input / output spaces $\mathcal{X}(K), \mathcal{Y}(K)$; it only enlarges the domain on which the layer acts. See Figure 5.



(a) (b) (c) (d)

Figure 5: Progression of topological domains with increasing modeling flexibility. **(a)** A graph represents only vertices and pairwise edges. **(b)** A simplicial complex adds higher-order simplices, but each simplex is determined by its lower-dimensional faces. **(c)** A cell complex allows more general cells, such as polygonal regions, whose boundaries need not be simplices. **(d)** A combinatorial complex further relaxes the construction by allowing independently specified, possibly overlapping higher-order relations. Each step increases the flexibility of the domain on which cochains and topological operators can be defined.

A.2 De Rham Correspondence and Physical Cochains

We recall the basic de Rham viewpoint used in the paper. The purpose is to explain why different physical quantities naturally live on different cell dimensions. Throughout, K denotes an oriented cell complex. We write K_k for the set of its k-dimensional cells: vertices for $k = 0$, edges for $k = 1$, faces for $k = 2$, and volumes for $k = 3$.

A k-*cochain* is a function assigning a number, or a vector of features, to each oriented k-cell:

$$C^k(K; \mathbb{R}^d) = \{u : K_k \to \mathbb{R}^d\}.$$

Thus a 0-cochain stores values on vertices, a 1-cochain stores values on edges, a 2-cochain stores values on faces, and so on.

The de Rham correspondence says that a smooth k-form ω should be discretized by integrating it over k-dimensional cells. This gives a cochain

$$\mathcal{I}_k(\omega)(\sigma) = \int_\sigma \omega, \qquad \sigma \in K_k.$$

Here $\mathcal{I}_k$ is the integration map from smooth k-forms to discrete k-cochains, and σ is an oriented k-cell. The intuition is simple: the degree of a field is determined by the kind of measurement it represents. Point values live on vertices, line integrals live on edges, surface fluxes live on faces, and volume densities live on volumes.

The exterior derivative is compatible with this discretization through Stokes' theorem:

$$\int_\sigma d\omega = \int_{\partial\sigma} \omega.$$

Here $d\omega$ is the continuum exterior derivative of ω, and $\partial\sigma$ is the oriented boundary of the cell σ. This identity is the continuum reason why the discrete derivative is built from boundary matrices.

Let

$$B_{k+1} : C_{k+1}(K) \to C_k(K)$$

be the oriented boundary matrix whose columns encode the oriented k-cell boundary of each $(k+1)$-cell. Its transpose gives the coboundary operator on cochains:

$$d^k : C^k(K) \to C^{k+1}(K), \qquad d^k = B_{k+1}^\top.$$

Intuitively, d^k takes a quantity measured on k-cells and produces its discrete variation on $(k+1)$-cells. For example, d^0 maps a scalar potential on vertices to edge differences, and d^1 maps edge circulations to face curls.

The codifferential moves in the opposite direction:

$$\delta^k : C^k(K) \to C^{k-1}(K).$$

It is the adjoint of d^{k-1} with respect to the discrete Hodge inner products. Therefore, unlike d^k, it depends not only on incidence but also on metric information, encoded by Hodge-star or mass matrices M_k. In matrix form one commonly writes

$$\delta^k = M_{k-1}^{-1} B_k M_k,$$

up to the sign convention chosen for the Hodge star. Intuitively, δ^k is the discrete divergence-type operator: it takes a quantity on k-cells and measures how it accumulates or flows into $(k-1)$-cells. See Figure 6.

The Hodge Laplacian combines the two directions:

$$\Delta_k = \delta^{k+1} d^k + d^{k-1} \delta^k.$$

It acts degree-wise,

$$\Delta_k : C^k(K) \to C^k(K),$$

and measures variation of a k-cochain through both its exterior derivative and its codifferential. For 0-cochains this recovers the usual graph or grid Laplacian. For higher-degree cochains it gives the corresponding edge, face, or volume Laplacian. See Figure 7 for an illustration.

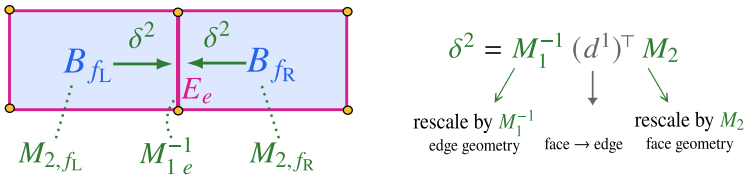


Figure 6: Codifferential as face-to-edge transport. For neighboring faces $f_{\mathrm{L}}, f_{\mathrm{R}} \in K_2$ sharing an edge $e \in K_1$, the codifferential $\delta^2 = M_1^{-1}(d^1)^\top M_2$ maps face quantities $B_{f_{\mathrm{L}}}, B_{f_{\mathrm{R}}}$ to the edge quantity E_e. Equivalently, its action on an edge is $(\delta^2 B)_e = (M_1^{-1})_{ee} \sum_{f \succ e} [f : e]\,(M_2)_{ff}\, B_f$. The incidence term $[f : e]$ determines which faces touch e and with what orientation, while M_2 and M_1^{-1} inject face and edge geometry.

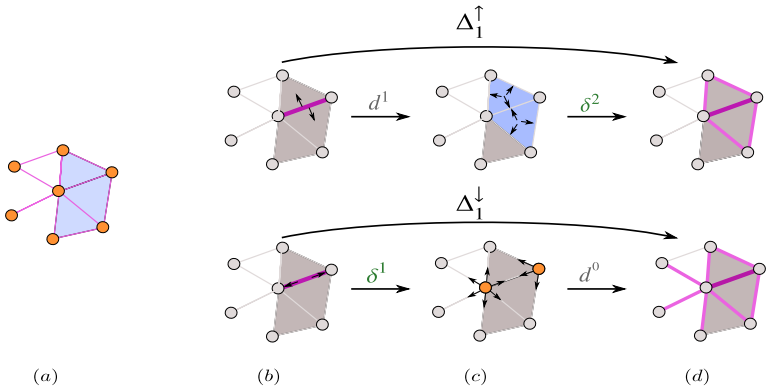


Figure 7: Upper and lower Hodge-Laplacian channels on edge cochains. Starting from an edge-supported signal in $C^1(K)$, the upper channel first applies $d^1 : C^1(K) \to C^2(K)$ to accumulate circulation on incident faces and then applies $\delta^2 : C^2(K) \to C^1(K)$ to return the result to edges, giving $\Delta_1^{\uparrow} = \delta^2 d^1$. This propagates information between edges through shared faces. The lower channel first applies $\delta^1 : C^1(K) \to C^0(K)$ to aggregate edge information to vertices and then applies $d^0 : C^0(K) \to C^1(K)$ to return it to edges, giving $\Delta_1^{\downarrow} = d^0 \delta^1$. This propagates information between edges through shared vertices. Together, $\Delta_1 = \Delta_1^{\uparrow} + \Delta_1^{\downarrow}$ decomposes same-rank edge propagation into face-mediated and vertex-mediated components.

This explains the structure of TNO layers. Many physical systems are not described by one type of field alone. Potentials, line integrals, fluxes, and densities naturally occupy different cochain spaces, while d, δ, and Δ provide the canonical operators that couple these spaces.

The table uses the primal de Rham convention. In an n-dimensional domain, the Hodge star identifies primal k-cochains with dual $(n-k)$-cochains. Consequently, the same physical vector quantity may be stored on different cells depending on whether one uses the primal or dual complex. Unless stated otherwise, this paper uses the primal convention in Table 4.

A.3 Continuum Physics and the mPDE Formulation

The governing equations of continuum physics describe how quantities living on different geometric dimensions interact. The electric field $\mathbf{E}$, integrated along oriented edges, and the magnetic flux $\mathbf{B}$, integrated over oriented faces, are therefore naturally a 1-cochain and a 2-cochain. Faraday's law, $\partial_t B = -d^1 E$, couples them through the exterior derivative d^1, which maps edge-valued data to face-valued data by accumulating signed values around each face. In physical terms, the circulation of $\mathbf{E}$ around a face boundary equals the negative rate of change of the magnetic flux through that face. Likewise, the incompressibility constraint $d^1 v = 0$ expresses divergence-freedom as a structural property rather than a penalty to enforce. More broadly, many physical laws are organized by the identity $d^{k+1} \circ d^k = 0$, which encodes, in a single algebraic statement, facts such as the absence of magnetic monopoles, the exactness of conservative fields, and the compatibility of flows. All are consequences of the same geometric principle: the boundary of a boundary is empty. The natural discrete setting is a cell complex K — vertices, edges, faces, and higher-dimensional cells with orientations — the minimal structure on which Stokes' theorem holds exactly. The theorem forces

Table 4: Physical quantities and their natural cochain degrees under the de Rham correspondence. A k-form is discretized as a k-cochain by integration over oriented k-cells. The table follows the primal convention in three dimensions.

Physical quantity	Geometric type	Degree k	Primal cell
Temperature, pressure, potential	0-form	0	Vertices
Electric field E, circulation, line flow	1-form	1	Edges
Magnetic flux B, vorticity flux, Darcy flux	2-form	2	Faces
Volumetric charge, mass, source density	3-form	3	Volumes

$B_k B_{k+1} = 0$, from which the exterior derivative $d^k = B_{k+1}^\top$, the codifferentials δ^k, and the Hodge Laplacians Δ_k all follow, forming the discrete exterior calculus on K — the t language in which physically coupled systems can be written and, therefore, the correct domain for learning their solution operators. On K, the governing laws of a broad class of physical systems define an operator

$$G : \bigoplus_i C^{k_i}(K; \mathbb{R}^{d_i}) \longrightarrow \bigoplus_j C^{\ell_j}(K; \mathbb{R}^{r_j}),$$

implicitly characterized by

$$\mathcal{F}_k\big(\sigma,\ u^k,\ d^{k-1}u^{k-1},\ \delta^{k+1}u^{k+1},\ \Delta_k u^k,\ a,\ f\big) = 0, \qquad \forall\, k,\ \forall\, \sigma \in K_k, \tag{mPDE}$$

where u^k are unknown cochains, a encodes material parameters at their natural degree, f a source, and $\mathcal{F}_k$ a possibly nonlinear local functional. The structure of (mPDE) is what makes the problem hard for existing architectures: the equation at degree k couples u^k to cochains one degree below via $d^{k-1}u^{k-1}$ and one degree above via $\delta^{k+1}u^{k+1}$ — cross-dimensional coupling that neither a grid nor a graph can express. Maxwell, incompressible Navier–Stokes, and linear elasticity are all instances of (mPDE); single-degree equations are the degenerate exception.

How this relates to operator learning. Operator learning aims to approximate such maps directly from data, without solving (mPDE) from scratch for every new configuration [43]. This perspective has produced a broad family of methods, including graph-based operators [46], Fourier-based operators [48], geometry-aware variants [45], implicit operators, attention-based methods [22, 36], and regional graph approaches [56]. Universal approximation results show that sufficiently expressive architectures can approximate continuous operators in suitable norms [43]. But for physical systems, approximation alone is not the whole story: the governing operator often carries algebraic structure that is itself physically meaningful. An architecture that does not represent this structure natively may still fit data well, but it must learn conservation, compatibility, and cross-degree coupling indirectly.

A.3.1 Example PDEs

We illustrate (Evol) with two standard systems whose fields live on different cochain degrees and couple through d and δ.

Example 1 (Discrete Maxwell-type coupling)**.** Maxwell's equations describe how electric and magnetic fields evolve together: a changing magnetic field induces an electric field, and a changing electric field, together with current, induces a magnetic field. On an oriented 3-complex K, the electric field is naturally represented as an edge cochain $E \in C^1$, while the magnetic flux is represented as a face cochain $B \in C^2$. The core discrete coupling has the form

$$\partial_t B = -d^1 E, \qquad \partial_t E = \delta^2 B - J,$$

with constraints

$$d^2 B = 0, \qquad \delta^1 E = \rho.$$

Thus E drives B through d^1, which measures circulation around faces, while B drives E through δ^2, which aggregates adjacent face fluxes onto edges. The essential point is not the specific physical normalization, but the cross-degree structure: fields living on different ranks of the complex interact through the exterior derivative and its adjoint.

Example 2 (Discrete wave-type coupling)**.** The wave equation describes how a disturbance propagates through an elastic medium: a displacement at one point pulls on its neighbors, which pull on theirs, and the wave travels outward. Let $u \in C^0$ be a scalar displacement on vertices, and let $p \in C^1$ be an

edge variable encoding local stretch or momentum-like flux. A first-order discrete wave system can be written schematically as

$$\partial_t u = \delta^1 p, \qquad \partial_t p = -c^2 d^0 u,$$

where c is the wave speed. Here d^0 sends vertex displacements to edge differences, while δ^1 aggregates edge quantities back to vertices. Eliminating p gives

$$\partial_t^2 u = -c^2 \delta^1 d^0 u = -c^2 \Delta_0 u,$$

showing that the wave operator itself is built from the same cross-degree maps.

B PDE Templates as Typed Cochain Systems

The previous sections define the main objects of the framework: a physical state is a tuple of cochains, the operators d, δ, Δ provide the allowed transport between and within degrees, and a TNO layer learns how to mix the transported features without learning the incidence support itself. The purpose of this section is to make this correspondence explicit for standard PDE systems.

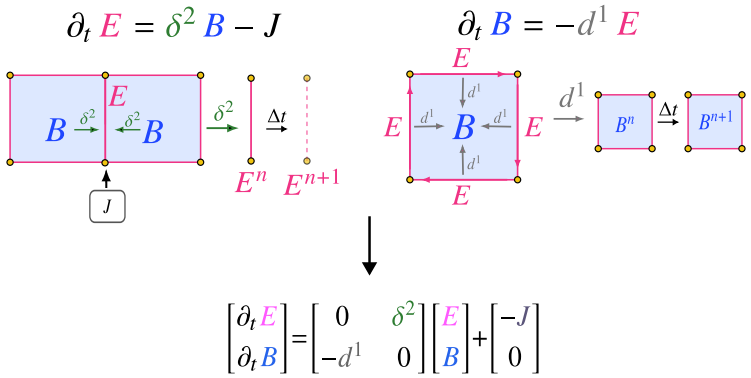


Figure 8: Maxwell coupling as cross-rank cochain transport. The electric field E lives on edges and the magnetic flux B lives on faces. Faraday's law uses d^1 to map edge circulation to face updates, while the Ampère–Maxwell law uses δ^2 to map neighboring face fluxes back to edge updates, with source J. Together, these two typed updates form the block Maxwell system.

A useful example to explain this correspondence is Maxwell's equations illustrated in Figure 8. In a compatible discretization, the electric field is not naturally a scalar value at vertices; it is a circulation along oriented edges. Likewise, the magnetic field is a flux through oriented faces. Thus

$$E \in C^1(K), \qquad B \in C^2(K).$$

The discrete curl is the coboundary

$$d^1 : C^1(K) \to C^2(K),$$

which sends edge circulations to face fluxes. Its metric adjoint,

$$\delta^2 : C^2(K) \to C^1(K),$$

sends face fluxes back to edge-supported quantities. Ignoring material weights for notational clarity, the semi-discrete Maxwell system has the typed form

$$\partial_t B = -d^1 E, \qquad \partial_t E = \delta^2 B - J,$$

or, equivalently,

$$\partial_t \begin{bmatrix} E \\ B \end{bmatrix} = \begin{bmatrix} 0 & \delta^2 \\ -d^1 & 0 \end{bmatrix} \begin{bmatrix} E \\ B \end{bmatrix} - \begin{bmatrix} J \\ 0 \end{bmatrix}.$$

This is precisely the kind of coupling encoded by the TNO block: the equation does not merely say that two feature arrays interact; it specifies that one field lives on edges, the other on faces, and that

Table 5: PDE templates as typed cochain systems. The DEC column shows the structural skeleton of the equation, while the TNO column shows the corresponding learned nonlinear cochain map.

PDE / system	**DEC core structure**	**TNO generalization**	**Practical challenge / insight**
Poisson / diffusion	$\Delta_0 u = f, \quad \Delta_0 = \delta^1 d^0$	$H^0_{\mathrm{out}} = \phi_\theta(H^0, \Delta_0 H^0 W_\Delta, f)$	Learns a Green-type solution operator on irregular meshes.
Heat equation	$\partial_t u + \Delta_0 u = f$	$H^0_{t+\Delta t} = \phi_\theta(H^0_t, \Delta_0 H^0_t W_\Delta, f_t, t)$	Time enters as a feature, recurrent state, or learned time-step map.
Advection–diffusion reaction	$\partial_t c + \mathcal{L}_v c = \kappa \Delta_0 c + R(c)$	$H^0_{\mathrm{out}} = \phi_\theta(H^0, \mathcal{L}_v H^0, \Delta_0 H^0 W_\Delta, R_\theta(H^0))$	Needs stable advection, upwinding, and control of stiff reactions.
Darcy / mixed elliptic	$q = -a\, d^0 p, \quad \delta^1 q = f$	$H^{0:1}_{\mathrm{out}} = \phi_\theta(\mathcal{B}_{01} H^{0:1} W, a, f)$	Pressure and flux are kept as different cochain types; conservation is native.
Conservation law	$\partial_t \rho + \delta^1 q = s$	$H^{0:1}_{\mathrm{out}} = \phi_\theta(H^0, H^1, \delta^1 H^1, s)$	Telescoping conservation follows from incidence algebra.
Wave, first-order	$\partial_t z = \mathcal{A}_{01} z, \quad z = (u, p)$	$H^{0:1}_{t+\Delta t} = \phi_\theta(H^{0:1}_t, \mathcal{A}_{01} H^{0:1}_t)$	Same cross-rank structure as Darcy, but used for evolution; energy matters.
Maxwell	$\partial_t z = \mathcal{A}_{12} z - (J, 0), \quad z = (E, B)$	$H^{1:2}_{\mathrm{out}} = \phi_\theta(\mathcal{B}_{12} H^{1:2} W, J)$	Separates electric circulation from magnetic flux; gauge and Gauss constraints matter.
Incompressible Navier–Stokes	$\partial_t u + \mathcal{L}_u u = -\nabla p + \nu \Delta_1 u, \quad \delta^1 u = 0$	$H^1_{\mathrm{out}} = \Pi_{\delta=0} \phi_\theta(H^1, \mathcal{L}_{H^1} H^1, \Delta_1 H^1 W_\Delta, \delta^2 H^2 W_\delta)$	Nonlinear advection must be paired with a hard divergence-free constraint.
Compressible Euler / NS	Continuity, momentum, and energy equations with fluxes	$H^{0:2}_{\mathrm{out}} = \phi_\theta(H^{0:2}, dH, \delta H, \mathcal{F}_\theta(H^{0:2}))$	Requires shock-aware fluxes, positivity, and entropy consistency.
Elasticity	$\operatorname{div} \sigma = f, \quad \sigma = \mathbb{C} : \varepsilon(u), \quad \varepsilon(u) = \frac{1}{2}(d^0 u + (d^0 u)^\top)$	$H^1_{\mathrm{out}} = \phi_\theta(H^1, \mathrm{sym}(d^0 H^0) W_\varepsilon, \Delta_1 H^1 W_\Delta, \mathbb{C})$	The symmetric gradient and constitutive law are the load-bearing structures.
Shallow water	$\partial_t h + \delta^1(hu) = 0,$ momentum balance	$H^{0:1}_{\mathrm{out}} = \phi_\theta(H^0, H^1, \delta^1(H^0 H^1), d^0 H^0, b)$	Couples height, flux, bathymetry, wetting/drying, and positivity.
Vorticity–stream	$\partial_t \omega + \{\psi, \omega\} = \nu \Delta_2 \omega, \quad \Delta_2 \psi = \omega$	$H^2_{\mathrm{out}} = \phi_\theta(H^2, \mathrm{Poisson}_\theta(H^2), \mathcal{L}_{H^1} H^2, \Delta_2 H^2 W_\Delta)$	The Poisson / Biot–Savart solve is the global component.
Mean-field games / OT	Continuity coupled with Hamilton–Jacobi–Bellman	$(\rho_{\mathrm{out}}, \varphi_{\mathrm{out}}) = \phi_\theta(\rho, \varphi, d^0 \varphi, \Delta_0 \varphi, \delta^1(\rho\, d^0 \varphi))$	Requires monotonicity, Wasserstein geometry, and stable density–potential coupling.
General k-form Hodge flow	$\partial_t u^k + \Delta_k u^k = f^k, \quad \Delta_k = \Delta_k^\uparrow + \Delta_k^\downarrow$	$H^k_{\mathrm{out}} = \phi_\theta(H^k, \Delta_k^\uparrow H^k W_\uparrow, \Delta_k^\downarrow H^k W_\downarrow, f^k)$	Universal form-valued PDE template; only the active operator block changes.

$$\mathcal{B}_{01} = \begin{bmatrix} \Delta_0 & \delta^1 \\ d^0 & \Delta_1^\downarrow \end{bmatrix}, \qquad \mathcal{B}_{12} = \begin{bmatrix} \Delta_1^\uparrow & \delta^2 \\ d^1 & \Delta_2^\downarrow \end{bmatrix}, \qquad \mathcal{A}_{01} = \begin{bmatrix} 0 & \delta^1 \\ d^0 & 0 \end{bmatrix}, \qquad \mathcal{A}_{12} = \begin{bmatrix} 0 & \delta^2 \\ -d^1 & 0 \end{bmatrix}.$$

the interaction must pass through the incidence operators of the complex. The corresponding learned cochain update has the same structural support,

$$\begin{bmatrix} H^1_{\mathrm{out}} \\ H^2_{\mathrm{out}} \end{bmatrix} = \phi_\theta\left(\begin{bmatrix} \Delta_1^\uparrow & \delta^2 \\ d^1 & \Delta_2^\downarrow \end{bmatrix} \begin{bmatrix} H^1 \\ H^2 \end{bmatrix} W,\ J \right).$$

Here H^1 and H^2 are hidden edge and face cochains. The block matrix fixes where information can flow: edge-to-face through d^1, face-to-edge through δ^2, and same-rank propagation through the

$$\begin{bmatrix} \partial_t E \\ \partial_t B \end{bmatrix} = \begin{bmatrix} 0 & \delta^2 \\ -d^1 & 0 \end{bmatrix} \begin{bmatrix} E \\ B \end{bmatrix} + \begin{bmatrix} -J \\ 0 \end{bmatrix} \qquad \begin{bmatrix} H^1_{\text{out}} \\ H^2_{\text{out}} \end{bmatrix} = \phi_\theta\left(\begin{bmatrix} \Delta_1^{\uparrow} W_1^{\Delta,\uparrow} & \delta^2 W_1^{\uparrow} \\ d^1 W_2^{\downarrow} & \Delta_2^{\downarrow} W_2^{\Delta,\downarrow} \end{bmatrix} \begin{bmatrix} H^1 \\ H^2 \end{bmatrix} \right)$$

(a) Typed Maxwell coupling. (b) Induced TNO layer.

Figure 9: From Maxwell coupling to a TNO template. **(a)** The Maxwell system is a typed cochain system: the electric field E lives on edges, the magnetic flux B lives on faces, and the PDE couples them through the cross-rank DEC maps $d^1 : C^1 \to C^2$ and $\delta^2 : C^2 \to C^1$, with source J acting on the edge equation. **(b)** The corresponding TNO layer keeps this typed routing. The off-diagonal blocks d^1 and δ^2 transport information between edge and face cochains, while the diagonal Hodge-Laplacian channels $\Delta_1^{\uparrow}$ and $\Delta_2^{\downarrow}$ perform same-rank propagation. Learning enters through the channel weights $W^{\bullet}$ and the nonlinear update ϕ_θ, which determine how the routed features are mixed rather than where information can flow.

Hodge Laplacian channels. The trainable part, W and ϕ_θ, learns how these transported features are combined. See Figure 9.

This example also shows the exact-sequence structure. Namely, the identity $d^2 d^1 = 0$ implies $\partial_t(d^2 B) = -d^2 d^1 E = 0$. Hence the magnetic Gauss constraint is preserved by the algebra of the complex. Similarly, $\delta^1 \delta^2 = 0$, together with the discrete charge-continuity law, gives the corresponding compatibility condition for the electric equation. These identities are inherited from the incidence structure, not learned as soft penalties.

Maxwell is only one instance of the general pattern. Poisson activates a single 0-cochain and the scalar Hodge Laplacian Δ_0. Darcy couples a 0-cochain pressure to a 1-cochain flux through d^0 and δ^1. Maxwell couples 1- and 2-cochains through curl-type operators. Fluid, elasticity, conservation-law, and transport systems add problem-specific nonlinearities such as advection, constitutive maps, learned fluxes, projections, or positivity constraints. In all cases, the same principle remains: the PDE determines the typed transport, while the TNO learns the nonlinear cochain map supported on that transport.

Table 5 summarizes this correspondence and should be read as an unpacking of the multi-degree PDE form and the block TNO layer, rather than as a collection of separate architectures.

C Proofs

C.1 Proof of Prop. 4.2 [FNO-Recovery]

Proof. On a uniform periodic cubical complex, the rank-0 Hodge Laplacian $\Delta_{0,N} = \delta^1 d^0$ is translation invariant and is diagonalized by the discrete Fourier basis:

$$\Delta_{0,N} = \mathcal{F}_N^{-1} \Lambda_N \mathcal{F}_N.$$

The zero eigenspace is the space of constant 0-cochains, hence

$$P_0^{\text{harm}} u = \mathcal{F}_N^{-1}\big(\mathbf{1}_{\xi=0}\widehat{u}(\xi)\big), \qquad (I - P_0^{\text{harm}})u = \mathcal{F}_N^{-1}\big(\mathbf{1}_{\xi\neq 0}\widehat{u}(\xi)\big).$$

Define the rank-0 spectral TNO channel

$$S_{\theta,N} u = \mathcal{F}_N^{-1}\big(R_\theta(\xi)\widehat{u}(\xi)\big)_\xi$$

with the usual real-valuedness condition

$$R_\theta(-\xi) = \overline{R_\theta(\xi)}.$$

Equivalently, separating the harmonic and non-harmonic parts,

$$S_{\theta,N} u = \mathcal{F}_N^{-1}\big(\mathbf{1}_{\xi\neq 0} R_\theta(\xi)\widehat{u}(\xi)\big) + \mathcal{F}_N^{-1}\big(\mathbf{1}_{\xi=0} R_\theta(0)\widehat{u}(0)\big).$$

Thus

$$S_{\theta,N} u = S_{\theta,N}(I - P_0^{\text{harm}})u + R_\theta(0) P_0^{\text{harm}} u.$$

Now choose the rank-0 TNO layer

$$T_{\theta,N}(u) = W_{\text{self}} u + S_{\theta,N}(I - P_0^{\text{harm}})u + W_{\text{harm}} P_0^{\text{harm}} u,$$

with

$$W_{\mathrm{self}} = W, \qquad W_{\mathrm{harm}} = R_\theta(0).$$

Then

$$\begin{aligned} T_{\theta,N}(u) &= Wu + \mathcal{F}_N^{-1}\big(\mathbf{1}_{\xi\neq 0} R_\theta(\xi)\widehat{u}(\xi)\big) + \mathcal{F}_N^{-1}\big(\mathbf{1}_{\xi=0} R_\theta(0)\widehat{u}(0)\big) \\ &= Wu + \mathcal{F}_N^{-1} R_\theta \mathcal{F}_N u. \end{aligned}$$

This is exactly the linear FNO layer. □

C.2 Proof of Proposition 4.3 [GNO-Recovery]

Proof. A graph neural operator approximates an integral operator of the form

$$(\mathcal{K}_\theta u)(x) = \int_D \kappa_\theta(x,y)u(y)\,d\mu(y),$$

where $\kappa_\theta(x,y)$ is a learned kernel acting on the feature value at the source point y and producing a contribution at the query point x. On a finite point cloud $\{x_i\}_{i=1}^n$, this integral is replaced by a quadrature rule over a neighborhood $\mathcal{N}(i)$ of each query point:

$$(\mathcal{K}_\theta u)(x_i) \approx \sum_{j\in\mathcal{N}(i)} \kappa_\theta(x_i,x_j)u_j\omega_j,$$

where $\omega_j > 0$ is the quadrature weight associated with the source point x_j. A GNO layer then adds a pointwise channel map:

$$u_i \longmapsto Wu_i + \sum_{j\in\mathcal{N}(i)} \kappa_\theta(x_i,x_j)u_j\omega_j.$$

We now realize this update as a rank-0 graph-quadrature TNO. Construct a directed graph complex $G = (V,E)$ with one vertex v_i for each sample point x_i. For every quadrature interaction $j \in \mathcal{N}(i)$, add a directed rank-1 cell

$$e_{ij} : v_j \to v_i.$$

Thus each edge represents one source-to-target quadrature contribution.

Define a rank-1 cochain $h^1 \in C^1(G;\mathbb{R}^{d_h})$ by assigning to each directed edge the corresponding learned kernel message:

$$h^1(e_{ij}) = \kappa_\theta(x_i,x_j)u_j\omega_j.$$

This is a cellular message: it lives on the rank-1 cell e_{ij} and is supported only where the graph complex contains an incidence from source v_j to target v_i.

Let

$$S_{\mathrm{tar}} : C^1(G;\mathbb{R}^{d_h}) \to C^0(G;\mathbb{R}^{d_h})$$

denote the target-incidence aggregation operator

$$(S_{\mathrm{tar}}h^1)_i = \sum_{e_{ij}:\mathrm{tar}(e_{ij})=v_i} h^1(e_{ij}) = \sum_{j\in\mathcal{N}(i)} h^1(e_{ij}).$$

Equivalently, S_{tar} is the unsigned incoming-incidence matrix of the directed graph. It is important that this is the incoming-incidence aggregation, not the signed boundary operator: the signed boundary would also include source signs and would not equal the GNO quadrature sum in general.

Substituting the definition of h^1 gives

$$(S_{\mathrm{tar}}h^1)_i = \sum_{j\in\mathcal{N}(i)} \kappa_\theta(x_i,x_j)u_j\omega_j.$$

Therefore the rank-0 TNO update with self-channel $W_0^{\mathrm{self}} = W$ is

$$\mathcal{T}_{\theta,0}(u)_i = Wu_i + (S_{\mathrm{tar}}h^1)_i = Wu_i + \sum_{j\in\mathcal{N}(i)} \kappa_\theta(x_i,x_j)u_j\omega_j.$$

This is exactly the GNO graph-quadrature update. Applying the pointwise nonlinearity after this affine update gives the usual nonlinear GNO layer. □

C.3 Cell-wise Realization and Copresheaf Variant

The block update in Sec. 4.2 admits a cell-wise realization with the same DEC supports. For brevity, write $h^k_\sigma := h^k(\sigma)$. For a k-cell σ, define

$$\mathcal{N}^k_\uparrow(\sigma) = \{\tau \in \widetilde{K}_k : (\Delta^\uparrow_k)_{\sigma\tau} \neq 0\}, \qquad \mathcal{N}^k_\downarrow(\sigma) = \{\tau \in \widetilde{K}_k : (\Delta^\downarrow_k)_{\sigma\tau} \neq 0\}.$$

The four canonical messages are

$$m^{k,\downarrow}(\sigma) = \bigoplus_{\rho \prec \sigma} \psi^\downarrow_\theta\big(h^k_\sigma, h^{k-1}_\rho, [\sigma : \rho]\big)\,, \qquad m^{k,\uparrow}(\sigma) = \bigoplus_{\tau \succ \sigma} \psi^\uparrow_\theta\big(h^k_\sigma, h^{k+1}_\tau, (\delta^{k+1})_{\sigma\tau}\big)\,,$$

$$m^{k,\Delta^\uparrow}(\sigma) = \bigoplus_{\tau \in \mathcal{N}^k_\uparrow(\sigma)} \psi^{\Delta^\uparrow}_\theta\Big(h^k_\sigma, h^k_\tau, (\Delta^\uparrow_k)_{\sigma\tau}\Big)\,, \qquad m^{k,\Delta^\downarrow}(\sigma) = \bigoplus_{\tau \in \mathcal{N}^k_\downarrow(\sigma)} \psi^{\Delta^\downarrow}_\theta\Big(h^k_\sigma, h^k_\tau, (\Delta^\downarrow_k)_{\sigma\tau}\Big)\,.$$

The updated feature is

$$h^k_{\mathrm{out}}(\sigma) = \phi_\theta\Big(h^k_\sigma,\ m^{k,\downarrow}(\sigma),\ m^{k,\uparrow}(\sigma),\ m^{k,\Delta^\uparrow}(\sigma),\ m^{k,\Delta^\downarrow}(\sigma)\Big)\,.$$

Here $\bigoplus$ is permutation-invariant. The downward message uses oriented incidence, while the upward and Hodge-Laplacian messages carry the metric information through the coefficients of δ and Δ.

D TNOs, Hodge-Compatible Smoothing, and the Multigrid Correspondence

This appendix explains the numerical-analysis interpretation of TNO and HTNO. The point is not that a trained TNO inherits the convergence theory of classical multigrid. It does not. Rather, the claim is structural: the four channels of the TNO layer, the use of residual depth, and the hierarchical construction of HTNO reproduce the algebraic ingredients that make multigrid work for de Rham complexes. These are precisely the ingredients missing from ordinary point smoothers on vector-valued nodal fields.

D.1 Why scalar multigrid does not directly extend

For the scalar Poisson problem

$$-\Delta_0 u = f, \qquad u \in C^0(K),$$

geometric multigrid has a simple mechanism. A local smoother damps oscillatory error on the fine grid; restriction moves the remaining low-frequency error to a coarse grid; prolongation returns the correction. Under standard assumptions, the convergence rate is independent of mesh size [71, 18].

This picture changes for de Rham systems. Consider the curl–curl block

$$\delta^2 d^1 e + \tau e = f, \qquad e \in C^1(K),$$

which appears in time-harmonic Maxwell equations, magnetostatic vector potentials, and the degree-one Hodge Laplacian. The semidefinite part $\delta^2 d^1$ has a large nullspace. Since

$$d^1 d^0 = 0,$$

every exact edge cochain $d^0\phi$ lies in the kernel of $\delta^2 d^1$. Thus the kernel contains $\operatorname{im} d^0$, of dimension $\dim C^0 - \beta_0$, and on topologically nontrivial domains it also contains harmonic representatives. A smoother acting only on C^1 through the curl–curl energy cannot see these components. More sweeps do not fix the problem; the iteration is blind to part of the error.

The same obstruction appears at every intermediate degree. For

$$\Delta_k = \delta^{k+1} d^k + d^{k-1} \delta^k,$$

the error decomposes into exact, coexact, and harmonic parts. A local smoother on C^k alone does not control all three. This is the basic reason that standard scalar multigrid does not transfer unchanged to $H(\mathrm{curl})$, $H(\mathrm{div})$, or general Hodge–Laplacian problems [37, 38, 4, 3].

D.2 Hiptmair smoothing

Hiptmair's remedy is to smooth not only on the space of the unknown, but also on the neighboring spaces in the de Rham complex. For an edge variable $e \in C^1(K)$, the smoother combines:

1. a relaxation on C^1, which acts on the coexact component;
2. an auxiliary relaxation on C^0, lifted by d^0, which acts on the exact component;
3. a harmonic correction when $\beta_1 > 0$.

This matches the discrete Hodge decomposition

$$C^1(K) = \operatorname{im} d^0 \oplus \mathcal{H}^1 \oplus \operatorname{im} \delta^2.$$

At degree k, the same principle uses the neighboring spaces C^{k-1} and C^{k+1} together with the harmonic subspace $\mathcal{H}^k = \ker \Delta_k$. The lesson is simple but important: a Hodge-compatible smoother for k-cochains must communicate with adjacent cochain degrees through the discrete differential and codifferential. This is not an implementation detail; it is what removes the kernel blindness of ordinary local relaxation.

D.3 The TNO layer as a Hodge-structured update

The linear TNO layer at degree k has the form

$$T_{\theta,k}(h) = d^{k-1}h^{k-1}W_k^{\downarrow} + \delta^{k+1}h^{k+1}W_k^{\uparrow} + P_k^{\mathrm{harm}}h^k W_k^{\mathrm{harm}} + h^k W_k^{\mathrm{self}}.$$

The first three terms land in the three Hodge components of C^k:

$$\operatorname{im} d^{k-1}, \qquad \operatorname{im} \delta^{k+1}, \qquad \mathcal{H}^k = \ker \Delta_k.$$

The final term is a local residual channel on the full space C^k.

The branch

$$d^{k-1}h^{k-1}$$

moves lower-degree information into degree k through the exact component. This is analogous to auxiliary-space corrections in Hiptmair-type smoothers, where a lower-degree potential correction is lifted into the target cochain degree. The branch

$$\delta^{k+1}h^{k+1}$$

moves higher-degree information into degree k through the coexact component. The harmonic branch

$$P_k^{\mathrm{harm}}h^k$$

extracts the component not seen by either d^k or δ^k, while the self-channel provides a local update on C^k.

Thus the layer has the same typed correction paths used by Hodge-decomposition-based smoothers, but with learned channel maps in place of fixed scalar relaxation parameters. This is a structural analogy rather than a multigrid convergence statement: a classical smoother updates solver residuals, whereas a TNO layer updates learned cochain features.

D.4 Depth as preconditioned iteration

The residual TNO update

$$h_{\ell+1}^k = h_\ell^k + H_\theta^k\big(h_\ell^{k-1}, h_\ell^k, h_\ell^{k+1}, d^{k-1}h_\ell^{k-1}, \delta^{k+1}h_\ell^{k+1}, \Delta_k h_\ell^k, a, f\big)$$

has the form of a stationary residual iteration. If A is the discrete PDE operator and B_θ is the learned Hodge-compatible preconditioner encoded by the layer, then the idealized update is

$$h_{\ell+1} = h_\ell + B_\theta^{-1}(f - Ah_\ell).$$

Stacking L residual layers is therefore analogous to applying L steps of a preconditioned Richardson iteration. This gives a concrete interpretation of depth. It is not merely additional expressivity; it is additional correction time. The diminishing returns observed with increasing depth are consistent with this solver interpretation: once the dominant residual components have been reduced, further iterations contribute less.

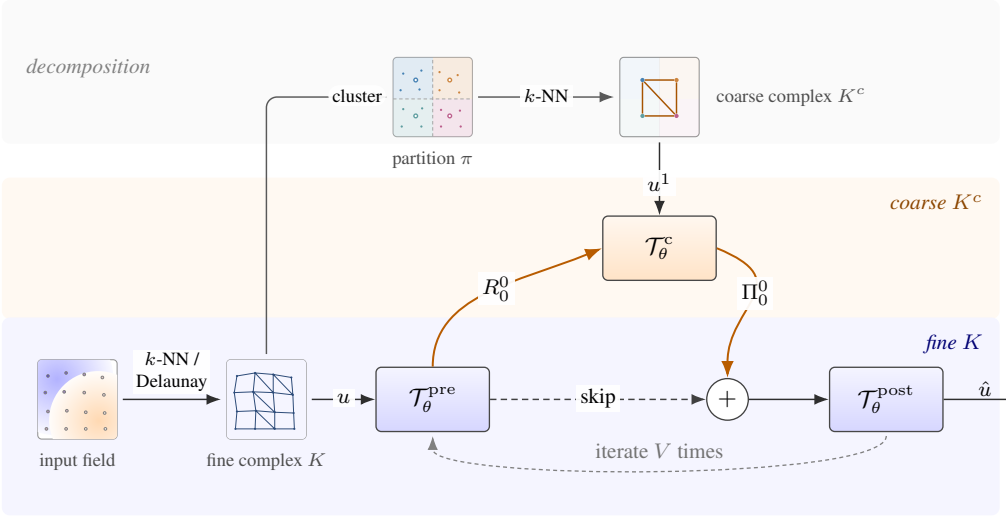


Figure 10: HTNO as a learned two-grid V-cycle. **Bottom:** the input field on a point cloud is lifted to a fine cell complex K (k-NN or Delaunay), giving an input cochain $u \in C^\bullet(K)$ that drives the V-cycle. The pre-smoother $\mathcal{T}_\theta^{\mathrm{pre}}$ runs on K; its output is restricted by R_0^0 to the coarse complex, where $\mathcal{T}_\theta^{\mathrm{c}}$ computes a correction; the correction is prolonged back by Π_0^0 and added to the pre-smoothed state via the skip; $\mathcal{T}_\theta^{\mathrm{post}}$ closes the cycle, producing $\hat{u}$. The whole block can be iterated V times with shared weights. **Top:** the coarse complex is built once from K. Fine vertices are clustered (fixed k-means or learned soft Voronoi) into a partition π, and k-NN on the resulting centroids yields the coarse complex K^c, on which the coarse cochain u^1 lives.

D.5 HTNO as a learned two-grid method

HTNO adds a compatible coarse complex K^c and degree-wise transfer maps

$$R_k : C^k(K) \to C^k(K^c), \qquad \Pi_k : C^k(K^c) \to C^k(K).$$

For de Rham systems, the key requirement is that transfer commute with the coboundary:

$$d^k_{K^c} R_k = R_{k+1} d^k_K, \qquad d^k_K \Pi_k = \Pi_{k+1} d^k_{K^c}.$$

This is the standard commuting-diagram condition in FEEC multigrid [4, 3]. It ensures that exact cochains remain exact across levels and that the discrete differential structure is not destroyed by restriction or prolongation. Without this condition, a coarse correction can turn a gradient-like error into a non-gradient error after prolongation, breaking the decomposition on which the smoother relies.

With commuting transfers, an HTNO block is a learned two-grid V-cycle (Fig. 10) [71, 19] that pre-smooths on K with Hodge-compatible TNO layers [37, 38], restricts via R_k, computes an approximate correction on K^c with coarse-level TNO layers, prolongs via Π_k, additively adds the correction to the pre-smoothed state, and post-smooths on K. With $L = 2$ this reads

$$\mathrm{HTNO}_\theta(h) \;=\; \mathcal{T}_\theta^{\mathrm{post}}\Big(\mathcal{T}_\theta^{\mathrm{pre}}(h) \;+\; \Pi_0^0\, \mathcal{T}_\theta^{\mathrm{c}}\big(R_0^0\, \mathcal{T}_\theta^{\mathrm{pre}}(h)\big) \Big). \tag{9}$$

Our implementation realises R_0^0, Π_0^0 as mean-pool and broadcast over a partition $\pi : V(K) \to \{1, \dots, K_c\}$ of the fine vertices. The partition is either *fixed* (k-means on input coordinates, SLIC superpixels, or an MS-complex segmentation), or *learned end-to-end* as a soft Voronoi diagram: K_c trainable query vectors attend over the warm-up features to predict centroid positions, soft assignments are obtained from $W = \mathrm{softmax}(-\|p - c\|^2/T)$ (or a Gaussian RBF kernel), and auxiliary spread and entropy losses on W keep the centroids from collapsing. The coarse complex K^c is built by k-NN over the centroids, inheriting its own incidence matrices and Hodge stars. Higher-rank transfers ($k \geq 1$) would require Whitney/Nédélec-style structured prolongation [57, 15]; we sidestep this by re-deriving rank-1 and rank-2 features on K^c from the pooled vertex field via the coarse incidence, so $\mathcal{T}_\theta^{\mathrm{c}}$ runs the multi-rank TNO update of (7) on K^c when its higher-rank features are present and a

rank-0 message-passing block otherwise. The fine–coarse–fine block used in our experiments is the $V = 1$ instance of (9); iterating with shared weights for $V > 1$, or recursing on deeper hierarchies, gives learned multilevel V-cycles.

D.6 Scope of the analogy

The correspondence with Hodge-compatible smoothing is structural, not a multigrid convergence theorem. Classical multigrid convergence depends on a specified residual equation, smoother, transfer operators, and regularity assumptions. A TNO layer does not inherit these rates automatically. What is shared is the algebraic form: cross-degree transport is fixed by d^{k-1} and δ^{k+1}, harmonic components are accessed through P_k^{harm}, and inter-level maps can be chosen to commute with d. Training then learns channel mixing, nonlinear correction, and coarse representations inside this fixed de Rham structure.

This distinction is also what separates the construction from point-only neural operators. A point-only model may use pooling, attention, U-Net skips, or learned coarse variables, but it does not by default carry typed cochain spaces C^k, fixed maps $d^k : C^k \to C^{k+1}$, codifferentials δ^{k+1}, or harmonic projectors. TNOs make these objects primitive operators of the architecture rather than behaviors to be inferred from data.

E Implementation Details

E.1 MPNN ablation baseline

The MPNN used in the ablations of Sec. 5.4 is a standard residual message-passing network on rank-0 (vertex) features only, sharing the Delaunay graph and the node/edge inputs of TNO but with no sheaf, harmonic, or higher-rank channels. Inputs $x_v \in \mathbb{R}^{F_{\mathrm{in}}}$ and edge features $e_{ij} \in \mathbb{R}^{F_e}$ are lifted to width h by a $\mathrm{Dense} \to \mathrm{swish} \to \mathrm{LayerNorm}$ encoder applied independently on vertices and edges. Each of the $L{=}12$ residual layers, with $\bar{h} = \mathrm{LN}(h)$, computes

$$m_{ij} = \mathrm{swish}\big(W_m\,[\,\bar{h}_i \,\|\, \bar{h}_j \,\|\, e_{ij}\,]\big), \qquad a_i = \sum_{j:\,j\to i} m_{ij}, \tag{10}$$

$$h_i \;\leftarrow\; h_i \;+\; \mathrm{LN}\Big(\mathrm{swish}\big(W_2\,\mathrm{swish}(W_1\,[\,\bar{h}_i \,\|\, a_i\,])\big)\Big), \tag{11}$$

with sum aggregation on the destination index. A final $\mathrm{LN} \to \mathrm{Dense}$ decoder produces the vertex output. The matched-width baseline uses $h{=}192$, the parameter-matched baseline uses $h{=}272$ (cf. Sec. F.5); all other optimizer/training settings are inherited from the ablation backbone.

Residual layers and physical structure. When the target $\mathcal{G}$ arises from a discrete PDE, the architecture should reflect the structure of that law. Rather than learning d^k, δ^k, Δ_k from scratch, a *residual TNO layer* uses them as fixed scaffolding and learns corrections. Writing $\mathbf{H}_\ell = (H_\ell^0, \ldots, H_\ell^N)$ for the hidden cochains at layer ℓ on the lifted complex $\widetilde{K}$, the residual form of $\mathbf{H}_{out}$ (as in Sec. 4.2) is

$$\mathbf{H}_{\ell+1} = \mathbf{H}_\ell + \phi_\theta\big(\mathcal{D}\,W \,\star\, \mathbf{H}_\ell\big), \tag{12}$$

where $\mathcal{D}$ is the block (Dirac-type) operator in Sec. 4.2 [10, 21], with Δ_k on the diagonal and d^{k-1}, δ^{k+1} on the off-diagonals. Coefficient and source cochains a, f enter as additional input channels at the appropriate ranks, consistent with (P1). Per rank, this expands to

$$H_{\ell+1}^k = H_\ell^k + \phi_\theta\Big(H_\ell^k,\; d^{k-1}H_\ell^{k-1}\,W_k^{\downarrow},\; \delta^{k+1}H_\ell^{k+1}\,W_k^{\uparrow},\; \Delta_k^{\uparrow}H_\ell^k\,W_k^{\Delta,\uparrow},\; \Delta_k^{\downarrow}H_\ell^k\,W_k^{\Delta,\downarrow}\Big), \tag{13}$$

with out-of-range terms omitted. The residual connection stabilizes training and makes the identity easy to represent, which is useful for small-Δt time-stepping operators.

Per-layer pseudocode. Alg. 1 unrolls one residual TNO layer in the form actually computed at runtime: each rank-k block is updated from its own state and the transported contributions of its neighboring ranks, with the supports prescribed by the DEC operators of $\widetilde{K}$. The block operator $\mathcal{D}$ of Sec. 4.2 already encodes the cross-rank routing, so no separate message-passing scaffolding is required: applying $\mathcal{D}$ to $\mathbf{H}_\ell$ is a single sparse block-matrix multiply that simultaneously realizes all three channels (exact, coexact, Laplacian) at every rank.

Input: Cochain features $\mathbf{H}_\ell = (H_\ell^0, \dots, H_\ell^N)$; DEC operators $\{d^{k-1}\}, \{\delta^{k+1}\}, \{\Delta_k^\uparrow\}, \{\Delta_k^\downarrow\}$ of $\widetilde{K}$; channel-mixing matrices $\{W_k^\downarrow, W_k^\uparrow, W_k^{\Delta,\uparrow}, W_k^{\Delta,\downarrow}\}$; feature transformation ϕ_θ
Output: Updated features $\mathbf{H}_{\ell+1} = (H_{\ell+1}^0, \dots, H_{\ell+1}^N)$
for $k = 0, 1, \dots, N$ **do**
  $m_k^\downarrow \leftarrow d^{k-1} H_\ell^{k-1} W_k^\downarrow$; // exact channel
  $m_k^\uparrow \leftarrow \delta^{k+1} H_\ell^{k+1} W_k^\uparrow$; // coexact channel
  $m_k^\Delta \leftarrow \Delta_k^\uparrow H_\ell^k W_k^{\Delta,\uparrow} + \Delta_k^\downarrow H_\ell^k W_k^{\Delta,\downarrow}$; // Hodge-Laplacian channel
  $H_{\ell+1}^k \leftarrow H_\ell^k + \phi_\theta\big(H_\ell^k, m_k^\downarrow, m_k^\uparrow, m_k^\Delta\big)$; // cf. Eq. (6)
end
return $\mathbf{H}_{\ell+1}$

Algorithm 1: Residual TNO layer (per layer ℓ, on lifted complex $\widetilde{K}$)

E.2 Complexity of a TNO layer

Let $n_k = |\widetilde{K}_k|$ be the number of rank-k cells, $n_\bullet = \sum_k n_k$, and $h = d_h$. Let

$$s_\bullet = \sum_{k=1}^{N} \mathrm{nnz}(B_k), \qquad \lambda_\bullet = \sum_{k=0}^{N} \Big(\mathrm{nnz}(\Delta_k^\uparrow) + \mathrm{nnz}(\Delta_k^\downarrow)\Big),$$

with nonzero counts taken for the sparse operators actually applied at runtime. A residual TNO layer has two costs. First, the DEC transports d^{k-1}, δ^{k+1}, and $\Delta_k^{\uparrow,\downarrow}$ are sparse matrix–dense matrix multiplies, costing $O(h(s_\bullet + \lambda_\bullet))$. Second, the learned maps are shared channel mixes applied cellwise, costing $O(n_\bullet h^2)$. Thus

$$T_{\mathrm{TNO}}^{(1)} = O\big(h(s_\bullet + \lambda_\bullet) + n_\bullet h^2\big)\,. \tag{14}$$

For bounded-degree cell complexes, $s_\bullet + \lambda_\bullet = O(n_\bullet)$, so a rigid DEC TNO layer is $O(n_\bullet h^2)$.

For the MPNN in Eqs. (10) and (11), let n_0 be the number of vertices, m the number of directed edges, and F_e the edge-feature dimension. The edge message MLP is evaluated on every edge, costing $O(mh(2h + F_e))$; aggregation costs $O(mh)$; and the node update costs $O(n_0 h^2)$. Hence

$$T_{\mathrm{MPNN}}^{(1)} = O\big((m + n_0)h^2 + mhF_e\big)\,. \tag{15}$$

For bounded-degree graphs this is $O(n_0 h^2)$. The comparison is therefore direct: both layers are linear in discretization size under bounded-degree sparsity.

E.3 Official Baseline Implementations: RIGNO, GAOT, HOGNN

For RIGNO [56], GAOT [75], and HOGNN [51], we use the authors' official implementations. These are adopted from published optimizer settings; we tune only the epoch budget and per-dataset width on the validation split; test numbers come from the best-validation checkpoint consistent with our models' evaluation protocol.

GAOT [75] is evaluated on both the RIGNO Tab. 1 and GAOT Tab. 2a benchmarks as it is the strongest published unstructured-mesh operator outside the RIGNO Table 1 lineup, and it shares our problem class. We follow the upstream optimizer (AdamW with cosine-decay, peak 10^{-3}, weight decay 10^{-5}, gradient clipping 1.0) and its effective batch of 64 (data-parallel over four GH200). We extend the epoch budget to 2,000 with patience 300 (the upstream configurations use 100 and 1,000 for PG and Elasticity, respectively); due to best-validation checkpoint selection, this remains a safety margin to assure convergence.

We note that the public GAOT release does not match the paper's Table 1 numbers on the GAOT suite itself (e.g. NACA0012 12.07% vs. paper 6.81%); this has been observed in issue #1 of the public repository that not all results are reproducible: for instance, they use 2048 train samples instead of 1024 (RIGNO / our protocol) on Poisson Gauss. GAOT entries are therefore a head-to-head comparison under identical experimental conditions, not as a reproduction of the paper's Table 1.

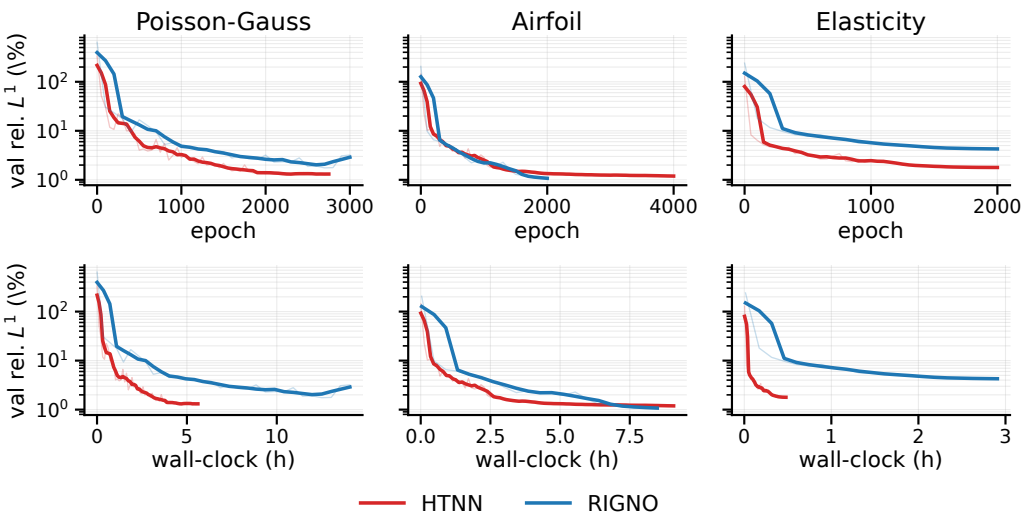


Figure 11: Validation relative L^1 vs. training epochs (left column) and cumulative wall-clock hours (right column) for HTNO and RIGNO on the three RIGNO steady-state benchmarks. Both models were trained under their respective official optimizer/batch settings; final test numbers correspond to the entries in Tab. 1.

RIGNO [56] convergence observations. In contrast to the RIGNO multi-level physical/regional graph, HTNN pools local regions into a coarse hierarchy, effectively capturing both short- and long-range information. In addition to a superior performance across numerous benchmarks (see Sec. 5), we observe notably faster convergence. Fig. 11 depicts two qualitative observations from training: on these three benchmarks, HTNO reaches lower validation error in notably fewer epochs and in less wall-clock time than RIGNO, except Airfoil, where the two are comparable. Field-level prediction comparisons for these benchmarks are given in Sec. G.1.

HOGNN [51] is a closely related baseline, in a spirit that shares motivations with TNO. Both lift signals beyond the vertices using DEC operators, providing an informative head-to-head comparison. We run it on every 2D benchmark with a matching TNO or HTNO entry: PG, AF, Elasticity, NACA0012/2412, RAE2822, PCS, and the synthetic-topology family (Sec. 5.4). We use the upstream optimizer (AdamW with cosine-decay, peak 5×10^{-4}, weight decay 10^{-4}, gradient clipping 1.0), 300 epochs, patience 200, and the identical Delaunay graph construction as for TNO, HTNN, and RIGNO; per-dataset tuning is restricted to hidden width. EmmiWing is excluded because the public release is 2D only, and porting its DEC operator stack to 3D surfaces is outside the scope of this work.

Table 6: HOGNN reproduction across all 2D benchmarks: test relative L^1 (%); lower is better, **bold**: best per column. "Best of ours" is the smaller of TNO and HTNO from the cited table.

	Tab. 1			Tab. 2a				Tab. 3a		Tab. 3b	
Method	Poisson-Gauss	Airfoil	Elasticity	NACA0012	NACA2412	RAE2822	PCS	projected	native rank-2	Darcy	Adv.-Diff.
Best of ours	**1.03**	**1.11**	**1.70**	**3.77**	**3.92**	**3.92**	**0.52**	**5.42**	**4.88**	**4.87**	**5.20**
HOGNN	33.78	15.64	21.24	9.36	10.33	9.28	50.44	14.84	15.97	12.04	13.23

Results. Tab. 6 lists test relative L^1 error for HOGNN on every benchmark with a matching TNO or HTNO entry. HOGNN does not match the leading method in any of the cells evaluated; we therefore consider it not well-suited for these domains, as the authors primarily study applications to electrostatics.

F Experimental Details

We provide additional implementation details for our models, and baselines. TNO and HTNO are implemented in JAX [17]. We use the author's official implementations (see Sec. E), which are often run in Pytorch [60]; we configure these with the same dataloaders our JAX models use to guarantee consistency. Implementations will be published upon acceptance. All models are run on a cluster of nodes consisting of 4 NVIDIA GH200s (96GB VRAM). The maximum compute budget on larger datasets (Emmiwing, PCS) is 4 GPUs simultaneously. Other steady-state benchmarks are run with 1 GPU per model.

F.1 Poisson-Gauss, Airfoil, Elasticity

Datasets. We use three datasets from the RIGNO benchmark [56]: **Poisson-Gauss** (PG, Poisson with Gaussian sources on a random point cloud, 9,216 nodes, $1024/128/256$ train/val/test, fixed positions); **Airfoil Flow** (AF, compressible Euler flow with varying airfoil geometry at fixed far-field M_∞=0.8, α=0° [48], ∼9K nodes on a 200×50 C-grid, $2048/128/256$, variable positions); **Elasticity** (hyper-elastic deformation of varying domains, ∼1K nodes, $1024/256/512$, variable). We note that AF is, on closer inspection, an effectively single-condition transonic benchmark; we audit its regime spread against the GAOT airfoils in Sec. G.2.

Baselines. We compare against the seven baselines reported in [56]: RIGNO-18, RIGNO-12, MeshGraphNet (MGN) [61], Geo-FNO [48], FNO DSE [52], GINO [49], and UPT [1]. We additionally evaluate GAOT [75] and HOGNN [51] on the same splits using the authors' official implementations (Sec. E.3).

Training protocol. For TNO, we report the best result from per-dataset tuning around a harmonic baseline (12 layers, hidden 128, K=8), varying depth, width, learning rate, neighborhood size, and Delaunay graph construction. HTNO uses dynamic k-means clustering (K=32–128 regions), 18 layers (6+6+6 fine/coarse/fine), hidden 128–256 depending on dataset scale. Training uses AdamW with cosine-decay (peak 5×10^{-4}), gradient clipping at 1.0, and early stopping with patience 200–500. Models train for 2,000–4,000 depending on convergence. The same protocol applies to the other benchmarks below.

F.2 Compressible Airfoils and Poisson-with-sines

Datasets. From the GAOT release [75] we use NACA0012/2412 and RAE2822 (∼8K-node airfoils, spanning subsonic, transonic, and supersonic regimes) and Poisson-C-Sines (PCS, fixed 16,431-node mesh).

Baselines. HTNO and TNO are trained as in the Sec. F.1 protocol; RIGNO and GAOT are evaluated on identical splits using the authors' official implementations (Sec. E.3).

Splits. The GAOT public release contains per-dataset NetCDF files but does not publish the train/val/test indices, making it challenging to reproduce Table 1. We thus reconstruct splits based on the sample counts declared in the paper. We follow the upstream data pipeline's default contiguous-block convention,

$$\text{train} = [0,\ N_{\text{tr}}), \quad \text{val} = [N_{\text{tr}},\ N_{\text{tr}} + N_{\text{val}}), \quad \text{test} = [N_{\text{tot}} - N_{\text{te}},\ N_{\text{tot}}),$$

i.e. the test split is the last N_{te} samples of the source file, with a gap between val and test on the larger airfoil files. Counts per dataset are listed in Tab. 7.

Table 7: Reconstructed GAOT-suite splits. N_{tot} is the sample count of the public release; PCS uses the maximal contiguous train slice the file admits.

Dataset	N_{tot}	N_{tr}	N_{val}	N_{te}
NACA0012	43,490	5000	128	256
NACA2412	47,925	5000	128	256
RAE2822	48,375	5000	128	256
Poisson-C-Sines	5,000	4616	128	256

Bluff-Body. The Bluff-Body benchmark from the same release is omitted: the public collection contains only 7 of the 10 paper pretraining shapes (Square, Cone, and Rectangle-L are absent, and the paper's 5-Ellipse pretraining set is replaced in the release by Ellipse-1/2 plus other shapes designated in the paper itself as fine-tuning geometries); we thus omit this evaluation due to incompleteness.

F.3 Large-Scale Surface PDEs: EmmiWing

Dataset and resolution protocol. EmmiWing [58] provides 29,727 steady-state compressible-RANS wing surfaces with raw meshes of 244K–426K nodes (variable across geometries). We use the official convex-hull-peeling split (ID-random, OOD-extrapolation, OOD-interpolation; $25{,}674/999/2{,}992$ train/val/test, with 62 erroneous cases excluded). Following AB-UPT [2], we adopt a $65{,}536$-geometry / $16{,}384$ sampling protocol: each raw mesh is once FPS-resampled to 65K shared query points, and a fresh $16{,}384$-node subset is drawn each training step. This follows established conventions in the literature: in [58], (PointNet, Transolver, ViT) also train at $16{,}384$ surface points; AB-UPT [2] uses the $65{,}536 \to 16{,}384$ supernode/anchor pattern that we follow most closely.

Deviations from the published protocol. We evaluate on the 65K FPS subset, where [58, 2] evaluate on the full 244K–426K raw mesh via chunked inference. Aggregate-L^1 numbers are therefore not directly comparable to the published Table 2 of [58]; the comparisons in Table 2b are between our four models trained and evaluated under the same regime. Closing this gap would require equipping the topological backbone with a neural-field-style decoder that decouples prediction resolution from the fixed shared mesh, which we leave as future work.

Training protocol. In addition to HTNO and RIGNO-18, we report Transolver [76] (the slice-attention transformer designed for unstructured meshes), a pre-norm self-attention Transformer [72] with sinusoidal coordinate-based positional encoding, and a PointNet-style baseline [63] (per-point MLPs with global max-pool, implemented as pre-norm residual blocks). All models are trained under matched compute budgets with AdamW, peak learning rate 5×10^{-4} on a cosine-decay schedule, gradient clipping at 1.0, and early stopping on validation L^1.

Per-channel results. Pressure is recovered to high fidelity by every model ($\leq 0.30\%$); the wall-shear-stress channels dominate the spread. HTNO leads on every channel; Transolver is the consistent runner-up, 0.2–0.3 pp behind on each τ component (see Tab. 2b).

F.4 Anisotropic Darcy with Per-Face Random Tensor Orientation

This section details the simulation protocol, dataset construction, and full result matrix for the rank-input study summarized in Sec. 5.3.

Synthetic-topology benchmark. The dataset family used in Sec. 5.4 is a 2D planar mesh distribution with a square outer boundary and $K \in \{0, 1, 2\}$ interior holes ($\sim$1,000 nodes per mesh) under standard boundary jitter and adaptive mesh-density variation. The data layout is 100 unique meshes $\times$ 50 samples per mesh (each dataset has $(4{,}000/500/500)$ train/val/test splits).

Anisotropic Darcy with face-valued K-tensor. We solve

$$-\nabla\cdot\big(K(x)\,\nabla u\big) = f, \qquad u|_{\partial\Omega_{\text{outer}}} = 0, \quad u|_{\partial\Omega_{\text{hole},k}} = g_k \sim \mathcal{U}[0.5, 1.5],$$

with $K(x)$ a DG0 (piecewise-constant per triangle) symmetric 2×2 tensor field reconstructed from a per-face orientation angle drawn iid per triangle, $\phi_t \sim \mathcal{U}[0, \pi)$:

$$K_{\text{face}}[t] \;=\; R(\phi_t)\,\mathrm{diag}(k_\parallel, k_\perp)\,R(\phi_t)^\top, \qquad k_\parallel{=}4,\; k_\perp{=}1 \text{ (anisotropy ratio 4)}.$$

Concretely $kxx[t]{=}k_\perp + (k_\parallel{-}k_\perp)\cos^2\phi_t$, $kyy[t]{=}k_\perp + (k_\parallel{-}k_\perp)\sin^2\phi_t$, $kxy[t]{=}(k_\parallel{-}k_\perp)\cos\phi_t\sin\phi_t$. The forcing f is a superposition of 1–3 signed Gaussian bumps with amplitudes $a_i \sim \mathcal{U}[1.0, 2.5]$, $\sigma{=}0.25$, and the same mesh-adaptive minimum-width constraint used in Sec. 5.4. The variational form is assembled in FEniCSx [7] with P_1 Lagrange for u and DG0 for the three K-tensor channels; the bilinear form expands the inner product directly to avoid UFL rank-mismatch on the asymmetric off-diagonal term.

Significance. The face channel $\cos(2\phi_t)$ has $\mathrm{Var}[\cos(2\phi_t)]{=}\frac{1}{2}$ per face under $\phi_t \sim \mathcal{U}[0, \pi)$, but population mean $\mathbb{E}[\cos(2\,\mathcal{U}[0, \pi))]{=}0$. The vertex-mean projection (incident-face average over 5–6

neighbors) therefore concentrates further toward 0 at every vertex: the projected channel is near-constant noise. Empirically the face channel has $\mathrm{mean}{=}-0.003$, $\mathrm{std}{=}0.696$, matching the analytic prediction. **Vertex projection is therefore lossy by construction**; the operator's coefficient field cannot be reconstructed from any vertex-supported representation.

Model input. Models receive $c{=}5$ vertex channels $[f, \not{*}_\partial, g, k_\parallel, k_\perp]$ (the last two are global scalars carried to every vertex), an edge channel $c_e \in \mathbb{R}^{|E|}$ (face-incident mean of $\cos(2\phi_t)$), and a face channel $c_f \in \mathbb{R}^{|F|}$ ($\cos(2\phi_t)$ per triangle). For TNOs, c_e is routed through the rank-1 cochain channel and c_f through the rank-2 channel. The *projected* experiments instead replace the rank-aux channels with their vertex-mean projection appended as channels 6–7 of the vertex-c feature, with native rank-aux ingestion disabled.

Training protocol. All cells share one backbone: standard TNO with sheaf transport, harmonic basis, hidden 192, dropout 0.20, AdamW with cosine-decay (peak $5{\times}10^{-4} \to 10^{-5}$, weight decay 10^{-4}), gradient clipping 1.0, batch size 16, 300 epochs. The two MPNN baselines do not have sheaf nor harmonic components; for parameter-matched experiments, MPNN uses a width $h{=}272$ to reach $\sim$5.35M parameters (vs. $\sim$5.0M for TNO).

F.5 Ablations

We instantiate two scalar PDEs whose flux carries non-trivial curl on this distribution: Darcy ($-\nabla{\cdot}(K\nabla u) + \sigma u = f$ with anisotropic K), and conservative advection–diffusion, both assembled in FEniCSx [7] with the same variational forms used in the rank-input study. Within each mesh block, only the per-hole Dirichlet BC values vary across the 50 samples (drawn $\sim \mathcal{U}[0.5, 1.5]$); the source field, diffusivity tensor, and advection scale are held fixed, so topology is controlled for.

Backbone. All ablation variants share one backbone: hidden dim 192, MLP messages, $r_{\max}{=}1$, dropout 0.20, AdamW with cosine-decay (peak $5{\times}10^{-4}$, weight decay 10^{-4}), gradient clipping at 1.0, batch size 16, 300 epochs, rigid 2D augmentation enabled, single GH200. Variants differ only in the sheaf-transport and the harmonic-basis input. The two MPNN baselines use the same backbone with sheaf and harmonic disabled; the param-matched MPNN uses width $h{=}272$ to reach $\sim$5.35M parameters (vs. 5.0M for TNO)

G Extended Experimental Results

This appendix collects qualitative field-level comparisons for the steady-state benchmarks of Sec. 5 and a per-dataset regime audit that explains why the TNO/RIGNO ranking inverts between the two airfoil families.

G.1 Qualitative comparisons on the steady-state suites

RIGNO-suite (Airfoil, Elasticity). Fig. 12 shows field-level predictions on a median-test-L^1 sample of each dataset for TNO, RIGNO, and HTNO. On AF, all three architectures recover the ground-truth density field to within $\sim$1–3% relative L^1, with residuals concentrated near the airfoil tip and along the wake; on Elasticity, the residuals localize on the loaded notch boundary. The visual closeness of the three error maps on AF mirrors the table-level finding (Tab. 1) that TNO and RIGNO-18 are largely indistinguishable on this benchmark. This can be attributed to the dataset's simple regime (see Sec. G.2).

GAOT-suite (PCS, NACA0012/2412, RAE2822). Fig. 13 shows the analogous comparison on the four single-file steady-state datasets of [75]. PCS (top row) is dominated by a high-frequency multiscale sinusoidal forcing pattern, which HTNO and GAOT can represent well. The three airfoil rows expose the regime-sweep difficulty of this benchmark: the operators concentrate residual along the boundary layer and—for the supersonic RAE2822 sample—along the captured shock, where flatter message-passing baselines visibly under-resolve the discontinuity while TNO's harmonic + DEC structure appears to track it more tightly. Per-row error panels share a common color scale, so RIGNO and HTNO residuals are directly comparable.

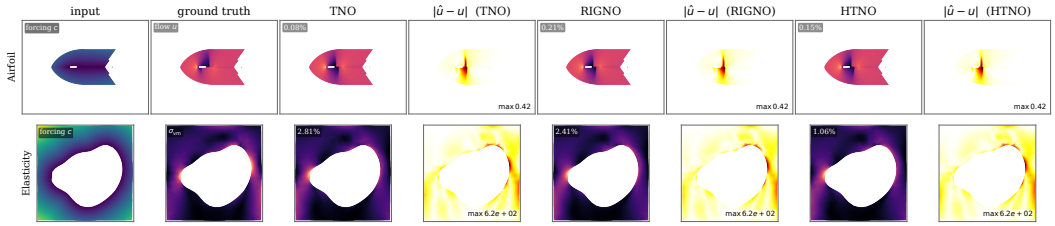


Figure 12: Qualitative comparison on the RIGNO-suite steady-state benchmarks (Airfoil, Elasticity). Each row shows one median-test-L^1 sample for the dataset; columns: input forcing, ground-truth field, and per-architecture prediction $\hat{u}$ followed by absolute error $|\hat{u} - u|$ for TNO, RIGNO, and HTNO. The three error panels in a row share a common color scale, with the shared $\max$ stamped on each panel so residual magnitudes are directly comparable across architectures.

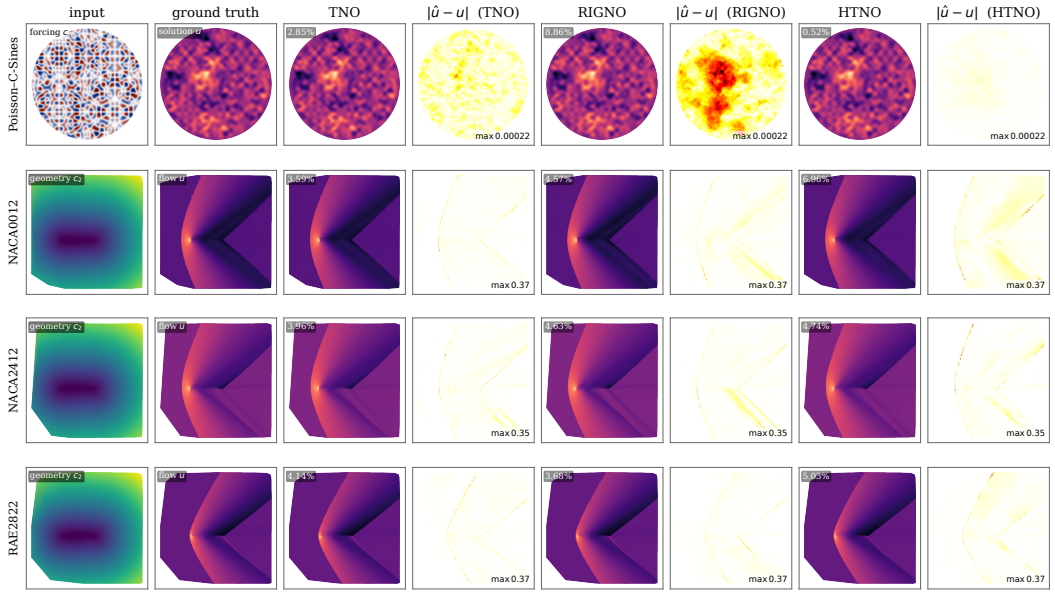


Figure 13: Qualitative comparison on the GAOT-suite benchmarks (Poisson-C-Sines, NACA0012, NACA2412, RAE2822). Each row shows one median-test-L^1 sample for the dataset; columns: model input, ground-truth field, and per-architecture prediction $\hat{u}$ followed by absolute error $|\hat{u} - u|$ for TNO, RIGNO, and HTNO. Per-row error panels share a common color scale, allowing residual magnitudes to be directly comparable across architectures. Triangulation on the airfoil rows uses a vertex-local long-edge filter to drop spurious convex-hull bridges across the wing void.

G.2 Differences in airfoils across datasets

TNO marginally underperforms RIGNO-18 on AF and outperforms it by 19–32% relative test-L^1 on the GAOT airfoils. This can be attributed to how the benchmarks differ in what their generators sweep across samples (Tab. 8): [48] fixes the AF far-field at M_∞=0.8, α=0° and varies only the airfoil geometry, exposing a single distance-to-boundary field as conditioning (RIGNO [56] adopts this dataset directly); [75] draws each NACA0012/2412/RAE2822 sample at a different (M, α) pair with $M \in [0.5, 1.4]$ and $\alpha \in [0.5°, 5.0°]$ on per-sample meshes, exposing M and α as conditioning scalars.

Table 8: What varies across samples for the two airfoil benchmark families (published quantities).

Dataset family	M_∞	α	conditioning channels	varies per sample
AF [48]	0.8 (fixed)	0° (fixed)	distance field	airfoil geometry only
GAOT airfoils [75]	$[0.5, 1.4]$	$[0.5°, 5.0°]$	M, α (scalars)	$(M, \alpha$, mesh)

H Related Work

Neural operators. Operator learning (OL) approximates maps between infinite-dimensional function spaces, with the aim of producing models that generalize across initial conditions, coefficients, forcings, and discretizations [43, 16, 42, 6]. *DeepONet* [55] and its physics-informed extensions [30, 41] learn a branch–trunk factorization of the operator with a universal approximation guarantee in function spaces. The *neural operator* family of Li et al. [46] instead realizes the operator through learned integral kernels: the Graph Neural Operator (GNO) approximates these kernels by message passing on sampled point clouds, while the Fourier Neural Operator (FNO) [47] replaces the kernel by a parameterized spectral multiplier on a regular grid, with subsequent factorized [68], wavelet [70], and convolutional [65] variants. Physics-informed neural networks complement these by deriving losses from the governing PDEs [50, 74, 79], and a separate line studies OL via model reduction [9].

Neural operators on irregular geometries. PDEs on irregular domains have motivated a body of operator-learning extensions beyond regular Cartesian grids. Geo-FNO [48] learns a deformation from the physical mesh to a uniform latent grid on which the FNO is applied; DAFNO [54] embeds geometry through smoothed indicator masks; and Beyond-Regular-Grids [52] evaluates truncated spectral transforms directly on arbitrary point sets. GINO [49] couples a graph encoder/decoder with an FNO on a uniform latent grid, scaling operator learning to large 3D meshes. Mesh-based simulators include MeshGraphNet [61], which propagates information on the simulation mesh together with a separate world-edge graph, and RIGNO [56], which uses message passing on multiscale graphs; GAOT [75] pairs a multiscale attentional graph encoder with a transformer processor on a latent grid. Transformer-based operators [45, 35, 76, 20, 22] treat unstructured nodes as tokens and attend either across all points or across learned cluster representatives, with universal physics transformers [1, 2] and PDE foundation models [36] extending this idea to large-scale 3D aerodynamics [58].

A complementary line treats geometry intrinsically: DIMON [77] learns the operator across a diffeomorphic family of domains, geometric NOs on Riemannian manifolds [64, 23] and gauge-equivariant intrinsic operators [25, 24] build manifold structure into the architecture, and Neural Green's Functions [78] parameterize geometry-dependent fundamental solutions. What unifies these methods is that all physical quantities are still represented as functions on *points* (graph nodes, deformed grid samples, or tokens); cross-dimensional differential structure is recovered indirectly through learned kernels. TNOs explicitly address this gap: by keeping cochains of all degrees as first-class objects and routing information through fixed boundary, coboundary, and Hodge operators, geometric type and the conservation identities $d \circ d = 0$ become structural rather than learned.

Discrete exterior calculus and structure-preserving discretizations. The mathematical foundation behind TNOs is the discretization of differential forms. Discrete exterior calculus [39, 29, 28, 27, 62, 8] models physical quantities as cochains on oriented cells and replaces gradient, curl, and divergence by signed boundary operators that satisfy a discrete Stokes' theorem. The same viewpoint underlies Whitney forms [14, 15], mixed and Nédélec finite elements [57, 38], mimetic finite differences [40, 53], and the finite element exterior calculus of Arnold et al. [4, 5], Arnold [3], all of which preserve the de Rham complex and the algebraic identity $B_k B_{k+1} = 0$ at the discrete level. TNOs inherit this structure: the cellular operators $d^k = B_{k+1}^\top$, δ^k, and the Hodge Laplacian Δ_k used in our layers are exactly the DEC operators on the underlying complex.

Topological deep learning (TDL) & architectures for PDE learning. TDL [59, 32] extends graph neural networks by allowing message passing across cells of multiple ranks (vertices, edges, faces, and higher). Simplicial complex networks generalize the Weisfeiler–Leman test to higher-order incidences [12], CW networks operate on regular cell complexes [11]. Combinatorial complexes [31] unify these under a single neighborhood scheme. Sheaf neural networks [34, 13] and copresheaf topological networks [33] attach learnable fiber maps to incidence relations, while Hodge Laplacian random walks [66, 73] and topology-aware GNNs [44], learning the hodge-star [67], provide foundations. Despite the appeal of the cellular viewpoint for physics, topological architectures remain almost absent from the OL literature, and the closest prior works are graph-based and message-passing in flavor: SNN-PDE [26] uses simplicial convolutions for **time-dependent** PDEs. Trask et al. Trask et al. [69] learn Whitney-form-style metric data on a fixed graph to obtain a data-driven exterior calculus that enforces conservation. Finally, DEC-HOGNN [51] builds a higher-order GNN on DEC and FEEC primitives for boundary-value electromagnetic problems. While these methods exploit cell-level features, **they remain GNN-style architectures**. None define true function-space operators, expose the Hodge decomposition as an architectural component or address multi-degree

mPDE systems where unknowns at different ranks couple simultaneously through d and δ. TNOs address each of these by fixing information flow to the DEC operators and by exposing the Hodge decomposition in the layer (Eq. (7)), so that exact, coexact, and harmonic channels carry physically meaningful components.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: claims of the proposed framework are thoroughly described throughout the paper.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: limitations of the proposed framework are addressed

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

Justification: Assumptions of the proposed framework are clearly stated during the preliminaries and methods section.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: details regarding implementation are provided. Code and additional experimental data will be released upon acceptance.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: yes, code and experimental data will be provided upon acceptance. Sufficient details for reproduction are provided in the supplementary at time of submission.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: yes, details are provided in the supplementary materials. Data manifests are generated for all datasets, which will be provided along with the official implementations.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: error bars are not provided. These experiments require extensive computing resources (often 4 GH200s for a single experiment), and are thus cost and environmentally intensive to run. The dataset sizes are sufficiently large (>4000 samples) such that significant deviations between runs are not observed.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Compute resources are explicitely listed.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: no special ethics considerations required; no, e.g., anonymity concerns.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: broader impact statement included in the concluding remarks.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.

- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: N/A

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: yes, existing methods / code run as baselins are properly credited where applicable.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: simulation data is well documented, reproducible, and will be shared upon acceptance.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: N/A

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: N/A

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.

- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs are not used in non-standard ways, e.g., as part of the method.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.